
COMPASS-SIG

Design Specifications and Supporting Documentation

Version: 1.2
Date: June 2026

Author: Geng Wang, Wenwen Xia, Zhedong Wang, Weidan Ji, Wenyu He, Ningyue Zhang, Fei Duan, Xinyang Geng, Jihong Li

Organization: Laboratory of Cryptology and Computer Security,
Shanghai Jiao Tong University; Shanghai Zhixun Crypto Co. Ltd.

Status: Draft Specification

Contents

1	Introduction	1
2	Algorithm Description	2
2.1	Symbols and Notations	2
2.2	Overview of the Signature Scheme	4
2.2.1	Algorithm Parameters	4
2.2.2	NTT domain representation	5
2.2.3	Hashing	5
2.2.4	Split and Compression	6
2.3	Core Algorithms	7
2.3.1	Key Generation	7
2.3.2	Signing	8
2.3.3	Verifying	10
3	Design Rationale	11
3.1	Security Foundation	11
3.2	Security Model	12
3.3	Parameter Sets	13
3.4	Failure Analysis and Number of Repetitions	14
3.5	Dissimilarity Claim	14
4	Security Claim and Analysis	15
4.1	Theoretical Security	15
4.2	Concrete Security	16
4.2.1	Concrete Security for Key Recovery	16
4.2.2	Concrete Security for Signature Forgery	19
4.2.3	Concrete Security for Strong Signature Forgery	22
5	Performances	23
5.1	Performances Analysis	23
5.1.1	Implementation Considerations and Tradeoffs	23
5.1.2	Performance Scaling Across Security Levels	24
5.2	Reference Implementation	24
5.2.1	Performance Results on x86 Architecture	24
5.2.2	AVX2 Optimized Implementation	26
6	Feature Statement	27

1. Introduction

This specification defines COMPASS-SIG, a part of the COMcompact Post-quANtum Security Suite, which is a post-quantum digital signature scheme (SIG) based on hard problems from module lattices. The algorithm is designed to be secure against large-scale quantum computers, and supports four security categories, corresponding to 128-bit (80-bit quantum), 256-bit (128-bit quantum), 384-bit (192-bit quantum), and 512-bit (256-bit quantum) security.

The design criteria of this algorithm are listed as follows:

- **The efficient and easy-to-implement Fiat-Shamir with abort paradigm.** Among all post-quantum signature paradigms (Fiat-Shamir with abort, hash-and-sign, MPC/VOLE-in-the-head, etc.), the Fiat-Shamir with abort (FSwA) paradigm is simple for implementation, with no complex submodules such as trapdoor generation or trapdoor sampling. Especially, when equipped with uniform-error LWE, FSwA enjoys a deterministic rejection sampling, that can be done efficiently within constant time, hence safe from side-channel attacks.

- **Avoiding the unnecessary redundancy in parameters.** The most famous FSwA lattice signature with deterministic rejection is the NIST standard ML-DSA (Dilithium). However, we point out that ML-DSA has unnecessary redundancy in its design, mainly from the conservativeness of the MSIS estimation. In ML-DSA, breaking MSIS only leads to a signature forgery, while breaking MLWE means a totally recovery of the secret key, that naturally implies a signature forgery. This is why the conservativeness in MSIS estimation has no impact on the security of ML-DSA, and in fact leads to a redundancy in the choice of parameters.

Instead of the conservative estimation as in ML-DSA, we present a novel design that offers a tight security estimation against signature forgery, i.e. for the hardness of MSIS/MISIS, and use our newly designed estimation tools for our security estimation. Additionally, we present a novel estimation for the security against strong unforgeability, and show that the additional condition for strong forgery is in fact harder to achieve than normal forgery, unlike what was indicated by the estimation result of ML-DSA. We note that some techniques used in our estimation were absent at the time when ML-DSA was proposed.

- **Optimizing the communication cost.** The design rationale of ML-DSA is that the size of public key + signature is minimized. Although the designers claimed that such requirement is rational in some cases such as certificate chains, in most protocols, each message contains either only one signature without public key, or a certificate (contains one public key and one signature) and an additional signature. So instead of minimizing the size of $pk + sig$, we further reduce the size of signature, at a cost of a larger public key. While our signature size is much smaller than ML-DSA, the size of $pk + 2 * sig$ is also smaller than ML-DSA, means that our scheme has less communication cost than ML-DSA under almost all cases.

To achieve such efficiency, compared to ML-DSA, we remove the usage of hints that allows a high compression rate to the public key. Although this leads to a larger public key size, we also avoid two disadvantages caused by introducing hints: first, including hint vector in the signature increases the signature size by about 5%;

second, the hint vector doubles the norm bound in MISIS, thus results in larger parameters.

- **Enhancing the security against signature forgery.** By observing the fact that all optimal lattice solving algorithms are under the Euclidean norm while our signature is under the infinity norm, we take a simple but significant approach to enhance the security of our signature against signature forgery, i.e. requiring the signature having a large enough Euclidean norm in addition to a small enough infinity norm, such that simply solving short lattice vectors under Euclidean norm can no longer lead to a signature forgery. The new hardness assumption is called Shell-MISIS (Shell-CVP), to show the fact that the required lattice vector lies in a shell between a high dimensional cube and a high dimensional ball.

We show that under this new hardness assumption, our scheme has similar security bits and less time cost compared with ML-DSA. But even if we do not consider the hardness of this new assumption, and only consider the security of signature forgery under MISIS, our scheme can still achieve the security levels required by the proposal.

2. Algorithm Description

2.1 Symbols and Notations

In this subsection, we specify the symbols, operators, and notational conventions used throughout the document. We define basic mathematical sets (e.g., $\mathbb{Z}$, $\mathbb{Z}_m$, $\mathcal{R}_q$), data types (e.g., bytes, byte strings), and operations (e.g., concatenation, rounding, NTT representation). We also establish typographic rules for distinguishing elements of different rings, vectors, matrices, and arrays, as well as conventions for indexing, subarrays, and typecasting. These notations form the precise language used in the algorithmic descriptions and security definitions that follow.

$a||b$ Concatenation of two bit/byte strings a and b .

$a \odot b$ Multiplication of a and b in the ring T_q .

$a \cdot b$ or ab Multiplication in any other rings.

$a + b$ Addition of a and b

a/b Division of integers. If b cannot be assumed to divide a , then either $\lfloor a/b \rfloor$ or $\lceil a/b \rceil$ is used.

$\perp$ Blank symbol that indicates failure or the lack of an output from an algorithm.

$\mathbb{B}$ The set $\{0, 1, \dots, 255\}$ of unsigned 8-bit integers (bytes).

$\mathbb{Q}$ The set of rational numbers.

$\mathbb{Z}$ The set of integers.

$\mathbb{Z}_m$ The ring of integers modulo m (i.e., the set $\{0, 1, \dots, m-1\}$ equipped with the operations of addition and multiplication modulo m).

- $\mathbb{Z}_m^n$ The set of n -tuples over $\mathbb{Z}_m$ equipped with $\mathbb{Z}_m$ -module structure. As a data type, this is the set of length- n arrays whose entries are in $\mathbb{Z}_m$.
- R The ring of single-variable polynomials over $\mathbb{Z}$ modulo X^n+1 , also denoted by $\mathbb{Z}[X]/(X^n+1)$.
- R_q The ring of single-variable polynomials over $\mathbb{Z}_q$ modulo X^n+1 , also denoted by $\mathbb{Z}_q[X]/(X^n+1)$, which consists of polynomials of the form $f = f_0 + f_1X + \dots + f_{n-1}X^{n-1}$, where $f_j \in \mathbb{Z}_q$ for all j .
- T_q The image of R_q under the number-theoretic transform. Its elements are called “NTT representations” of polynomials in R_q .
- S^* If S is a set, this denotes the set of finite-length tuples (or arrays) of elements from S , including the empty tuple.
- S^k If S is a set, this denotes the set of k -tuples (or length- k arrays) of elements from S .
- f_j The coefficient of X^j of a polynomial $f = f_0 + f_1X + \dots + f_{n-1}X^{n-1} \in R_q$.
- $\hat{f}$ The element of T_q that is equal to the NTT representation of a polynomial $f \in R_q$.
- $\mathbf{v}^T, \mathbf{A}^T$ The transpose of a row or column vector $\mathbf{v}$. In general, the transpose of a matrix $\mathbf{A}$.
- $B[i]$ The entry at index i in the array B . All arrays have indices that begin at zero.
- $B[k : m]$ The subarray $(B[k], B[k+1], \dots, B[m-1])$ of the array B .
- $|B|$ If B is a number, this denotes the absolute value of B . If B is an array, this denotes its length.
- $\lceil x \rceil$ The ceiling of x (the smallest integer greater than or equal to x).
- $\lfloor x \rfloor$ The floor of x (the largest integer less than or equal to x).
- $\lfloor x \rceil$ Rounding of x to the nearest integer. If $x = y + 1/2$ for $y \in \mathbb{Z}$, then $\lfloor x \rceil = y + 1$.
- $:=$ Denotes that the left-hand side is defined to be the right-hand side.
- $\leftarrow$ Denotes that the left-hand side is sampled as a uniform random element from the right-hand side.

Ring Operations. Elements of the rings $\mathbb{Z}, \mathbb{Z}_q, \mathbb{Z}_2, R$, and R_q are denoted by italicized lowercase letters (e.g., w). Elements of the ring T_q are length- n arrays of elements of $\mathbb{Z}_q$, and they are denoted by italicized letters with a hat symbol (e.g., $\hat{w}$). The addition and multiplication of elements of T_q are performed entry-wise. Thus, the i -th entry of the product of two elements $\hat{u}$ and $\hat{v}$ of T_q is $(\hat{u} \odot \hat{v})[i] = \hat{u}[i] \cdot \hat{v}[i] \in \mathbb{Z}_q$, where the multiplication operation in T_q is denoted by the symbol $\odot$.

When a product $a \cdot b$ or a sum $a + b$ is written and either a or b is a congruence class modulo m (i.e., if either a or b is an element of $\mathbb{Z}_m$ or R_m), then the product or sum is also understood to be a congruence class modulo m (i.e., an element of $\mathbb{Z}_m$ or R_m). Likewise, an element of R or $\mathbb{Z}$ may be “typecast” to an element of R_m or $\mathbb{Z}_m$, respectively, and may

be used as the input of a function specified to act on an element of R_m or $\mathbb{Z}_m$, respectively. In both cases, the element itself or its coefficients are mapped from $\mathbb{Z}$ to $\mathbb{Z}_m$ by taking the unique congruence class modulo m that contains the integer.

The coefficients of an element w of R or R_m are denoted by w_i so that $w = w_0 + w_1X + \dots + w_{n-1}X^{n-1}$. If w is in R (respectively, R_m) and t is in $\mathbb{Z}$ (respectively, $\mathbb{Z}_m$), then $w(t)$ denotes the polynomial evaluated at $X = t$.

Vectors with elements in R or R_m are denoted by bold lowercase letters (e.g., $\mathbf{v}$). Matrices with elements in R or R_m are denoted by bold uppercase letters (e.g., $\mathbf{A}$). If S is a ring and $\mathbf{v}$ is a length- L vector over S , then the entries in the vector $\mathbf{v}$ are expressed as $\mathbf{v}[0], \mathbf{v}[1], \dots, \mathbf{v}[L-1]$. The entries of a $K \times L$ matrix $\mathbf{A}$ over S are denoted as $\mathbf{A}[i, j]$, where $0 \leq i < K$ and $0 \leq j < L$.

The set of all length- L vectors over S is denoted by S^L . The set of all $K \times L$ matrices over S is denoted by $S^{K \times L}$. A length- L vector can also be treated as an $L \times 1$ matrix.

2.2 Overview of the Signature Scheme

2.2.1 Algorithm Parameters

This subsection defines the primary notation and parameters used in the algorithm. The main parameters of the algorithm are listed below.

- q : modulus of the integer ring $\mathbb{Z}_q$;
- n : degree of the polynomial modulus $X^n + 1$;
- k : number of MLWE samples;
- l : dimensions of MLWE secret;
- d : number of bits used for public key compression;
- η_s, η_e : infinity-norm bounds for the secret vectors $\mathbf{s}$ and $\mathbf{e}$;
- γ_1 : bound on the coefficients of the randomness vector y in signing;
- γ_2 : parameter defining the low-order bits when splitting (related to rounding);
- δ : the rescaling parameter, i.e. $\delta = \lfloor \gamma_2 / \gamma_1 \rfloor$;
- τ : number of non-zero entries in the challenge vector c' ;
- B : Euclidean norm bound for the short vector related to the signature $\mathbf{z}$.

Modular reductions. For an even (resp. odd) positive integer α , we define $r' = r \bmod^\pm \alpha$ to be the unique element r' in the range $-\frac{\alpha}{2} < r' \leq \frac{\alpha}{2}$ (resp. $-\frac{\alpha-1}{2} \leq r' \leq \frac{\alpha-1}{2}$) such that $r' \equiv r \pmod{\alpha}$. We will sometimes refer to this as a *centered* reduction modulo α . For any positive integer α , we define $r' = r \bmod^+ \alpha$ to be the unique element r' in the range $0 \leq r' < \alpha$ such that $r' \equiv r \pmod{\alpha}$. When the exact representation is not important, we simply write $r \bmod \alpha$.

Sizes of elements. For an element $w \in \mathbb{Z}_q$, we write $\|w\|_\infty$ to mean $|w \bmod^\pm q|$. We define the ℓ_∞ and ℓ_2 norms for $w = w_0 + w_1X + \dots + w_{n-1}X^{n-1} \in R$:

$$\|w\|_\infty = \max_i \|w_i\|_\infty, \quad \|w\| = \sqrt{\|w_0\|_\infty^2 + \dots + \|w_{n-1}\|_\infty^2}.$$

Similarly, for $\mathbf{w} = (w_1, \dots, w_k) \in R^k$, we define

$$\|\mathbf{w}\|_\infty = \max_i \|w_i\|_\infty, \quad \|\mathbf{w}\| = \sqrt{\|w_1\|^2 + \dots + \|w_k\|^2}.$$

We will write S_η to denote all elements $w \in R$ such that $\|w\|_\infty \leq \eta$. We will write $\tilde{S}_\eta$, to denote the set $\{w \bmod^\pm 2\eta : w \in R\}$. Note that S_η and $\tilde{S}_\eta$ are very similar except that $\tilde{S}_\eta$ does not contain any polynomials with $-\eta$ coefficients.

2.2.2 NTT domain representation

Our modulus q is chosen such that there exists a $2n$ -th root of unity r modulo q . This implies that the cyclotomic polynomial $X^n + 1$ splits into linear factors $X - r^i$ modulo q with $i = 1, 3, 5, \dots, 2n - 1$. By the Chinese remainder theorem our cyclotomic ring R_q is thus isomorphic to the product of the rings $\mathbb{Z}_q[X]/(X - r^i) \cong \mathbb{Z}_q$. In this product of rings it is easy to multiply elements since the multiplication is pointwise there. The isomorphism

$$a \mapsto (a(r), a(r^3), \dots, a(r^{2n-1})) : R_q \rightarrow \prod_i \mathbb{Z}_q[X]/(X - r^i)$$

can be computed quickly with the help of the Fast Fourier Transform. Since $X^n + 1 = X^n - r^n = (X^{n/2} - r^{n/2})(X^{n/2} + r^{n/2})$ one can first compute the map

$$\mathbb{Z}_q[X]/(X^n + 1) \rightarrow \mathbb{Z}_q[X]/(X^{n/2} - r^{n/2}) \times \mathbb{Z}_q[X]/(X^{n/2} + r^{n/2})$$

and then continue separately with the two reduced polynomials of degree less than $n/2$ noting that $X^{n/2} + r^{n/2} = X^{n/2} - r^{3n/2}$. The Fast Fourier Transform is also called Number Theory Transform (NTT) in this case where the ground field is a finite field. Natural fast NTT implementations do not output vectors with coefficients in the order $a(r), a(r^3), \dots, a(r^{2n-1})$. Therefore we define the NTT domain representation $\hat{a} = \text{NTT}(a) \in \mathbb{Z}_q^n$ of a polynomial $a \in R_q$ to have coefficients in the order as output by our reference NTT. Concretely,

$$\hat{a} = \text{NTT}(a) = (a(r_0), a(-r_0), \dots, a(r_{n/2-1}), a(-r_{n/2-1}))$$

where $r_i = r^{\text{brv}(n/2+i)}$ with $\text{brv}(k)$ the bitreversal of the $\log n$ bit number k . With this notation, and because of the isomorphism property, we have $ab = \text{NTT}^{-1}(\text{NTT}(a) \odot \text{NTT}(b))$. For vectors $\mathbf{y}$ and matrices $\mathbf{A}$, the representations $\hat{\mathbf{y}} = \text{NTT}(\mathbf{y})$ and $\hat{\mathbf{A}} = \text{NTT}(\mathbf{A})$ mean that every polynomial $\hat{y}_i$ and $\hat{a}_{i,j}$ comprising $\hat{\mathbf{y}}$ and $\hat{\mathbf{A}}$ is in NTT domain representation.

2.2.3 Hashing

The algorithm uses several different algorithms that hash strings in $\{0, 1\}^*$ onto domains of various forms. Below we give the high level descriptions of these functions.

Hashing to a Ball. Let B_τ denote the set of elements of R that have τ coefficients that are either -1 or 1 and the rest are 0 . We have $|B_\tau| = 2^\tau \cdot \binom{n}{\tau}$. For our signature scheme, we will need a cryptographic hash function that hashes onto B_τ in two steps. The first step consists of using a 2^{nd} pre-image resistant cryptographic hash function to map $\{0, 1\}^*$ onto the domain $\{0, 1\}^n$. The second stage consists of an XOF that maps the output of the first stage to an element of B_τ . The reason for this two-step process is that the output of the first stage allows for a very simple, compact description of an element in B_τ (under the assumption that the XOF is indistinguishable from a purely-random function). The algorithm we will use to create a random element in B_τ is sometimes referred to as an “inside-out” version of the Fisher-Yates shuffle, and its high-level description is in Fig. 1.¹ The XOF in the second stage is used to create the randomness needed for this algorithm (in Steps 03 and 04) using the output of the first stage as the seed.

```

SampleInBall $_\tau(\rho)$ 
01 Initialize  $\mathbf{c} = c_0 c_1 \dots c_{n-1} = 00 \dots 0$ 
02 for  $i := n - \tau$  to  $n - 1$ 
03    $j \leftarrow \{0, 1, \dots, i\}$ 
04    $s \leftarrow \{0, 1\}$ 
05    $c_i := c_j$ 
06    $c_j := (-1)^s$ 
07 return  $\mathbf{c}$ 

```

Figure 1: Create a random n -element array with τ ± 1 's and $n - \tau$ 0 's using the input seed ρ (and an XOF) to generate the randomness needed in Steps 03 and 04.

Expanding the Matrix $\mathbf{A}$. The function **ExpandA** maps a uniform seed $\rho \in \{0, 1\}^n$ to a matrix $\mathbf{A} \in R_q^{k \times l}$ in NTT domain representation. The matrix $\mathbf{A}$ is only needed for multiplication. Hence, for the sake of faster implementations, the expansion function **ExpandA** does not output $\mathbf{A} \in R_q^{k \times l} = (\mathbb{Z}_q[X]/(X^n + 1))^{k \times l}$. Instead it outputs $\hat{\mathbf{A}} \in T_q^{k \times l} = (\mathbb{Z}_q^n)^{k \times l}$, which is interpreted as the NTT domain representation of $\mathbf{A}$. As $\mathbf{A}$ needs to be sampled uniformly and the NTT is an isomorphism, **ExpandA** also needs to sample uniformly in this representation. To be compatible to COMPASS-KEM, an implementation whose NTT produces differently ordered vectors than our reference NTT needs to sample coefficients in a non-consecutive order.

Sampling uniform random vectors. The function **Sample**, takes as input an arbitrary set S and a seed ρ , outputs a uniform random element in S , i.e. $\mathbf{Sample}(S, \rho) := x \leftarrow S$.

2.2.4 Split and Compression

A splitting function splits an integer in $\mathbb{Z}_q$ into high-order bits and low-order bits. Given an integer $x \in \mathbb{Z}_q$ and a positive integer γ , the function $\text{Split}(x, \gamma)$ outputs a pair (x_1, x_0) such that

$$x = x_1 \cdot 2\gamma + x_0, \quad x_0 \in [-\gamma, \gamma].$$

¹Normally, the algorithm should begin at $i = 0$, but since there are $n - \tau$ 0 's, the first $n - \tau - 1$ iterations would just be setting components of $\mathbf{c}$ to 0 .

The goal of splitting is that when given an arbitrary element $r \in \mathbb{Z}_q$ and another small element $z \in \mathbb{Z}_q$, we would like to be able to recover the higher order bits of $r + z$ without knowing z . We can show that the following result holds:

Lemma 1. *For any integer $s \in \mathbb{Z}_q$ and any real $\beta > 0$, $(x_1, x_0) = \text{Split}(x, \gamma)$ and $(x'_1, x'_0) = \text{Split}(x + s, \gamma)$, if $|s| \leq \beta$ and $|x_0| < \gamma - \beta$, then $x_1 = x'_1$.*

Proof. $s = x + s - x = (x_1 - x'_1) \cdot 2\gamma + (x_0 - x'_0)$. Since $|x'_0| \leq \gamma$ and $|x_0| \leq \gamma - \beta - 1$, we have that $|x_0 - x'_0| \leq 2\gamma - \beta - 1$. If $x_1 \neq x'_1$, then $|x_1 - x'_1| \geq 1$. So $|(x_1 - x'_1) \cdot 2\gamma + (x_0 - x'_0)| \geq 2\gamma - (2\gamma - \beta - 1) = \beta + 1$, while $|s| \leq \beta$, which makes a contradiction. $\square$

The splitting function satisfying the condition above is not unique. In our algorithm, we specifically let $p = \lceil \frac{q}{2\gamma+1} \rceil$, set $x_1 = \lfloor \frac{p}{q} \cdot x \rfloor \bmod^+ p$, and $x_0 = x - x_1 \cdot 2\gamma \bmod^\pm q$.

For polynomial or polynomial vector inputs, Split is applied to each coefficient independently.

Compression and Decompression. To reduce the size of the public key, we introduce the following algorithms for compression and decompression. For an element $x \in \mathbb{Z}_q$, the compression function $\text{Comp}_d(x)$ is defined as:

$$\text{Comp}_d(x) = \begin{cases} \lfloor \frac{x-1}{2^d} \rfloor, & \text{if } x > 0, \\ \lfloor \frac{x}{2^d} \rfloor, & \text{otherwise.} \end{cases}$$

For a polynomial $x \in R_q$ (or a vector of polynomials), Comp_d is applied coefficient-wise.

The decompression function $\text{Decomp}_d(x)$ (for $x \in \mathbb{Z}_q$) returns

$$\text{Decomp}_d(x) = x \cdot 2^d + 2^{d-1},$$

and it is also applied coefficient-wise for polynomial and polynomial vector inputs.

2.3 Core Algorithms

This section describes the functions for key generation, signature generation, and signature verification. Where randomness is required, the random values are provided as inputs to the functions.

2.3.1 Key Generation

The key generation algorithm **COMPASS-SIG.KeyGen** takes a n -bit ($n/8$ -byte) random seed as input and outputs a public key and a private key that are both encoded as byte strings.

The seed **seed** is expanded as needed using an cryptographic hash function denoted by G to produce other random values. In particular,

- A $n/8$ -byte public random seed ρ . Using this seed, a polynomial matrix $\mathbf{A} \in R_q^{k \times l}$ is pseudorandomly sampled from $R_q^{k \times l}$.
- A $n/8$ -byte private random seed σ . Using this seed, the polynomial vectors $\mathbf{s} \in S_{\eta_s}^l$ and $\mathbf{e} \in S_{\eta_e}^k$ are pseudorandomly sampled from the subset of polynomial vectors whose coordinate polynomials have short coefficients.

The core cryptographic operation computes the public value

$$\mathbf{t} = \mathbf{A}\mathbf{s} + \mathbf{e}.$$

The vector $\mathbf{t}$ together with the matrix $\mathbf{A}$ may be considered an expanded form of the public key. The vector $\mathbf{t}$ is compressed in the actual public key by dropping the d least significant bits from each coefficient, thus producing the polynomial vector $\mathbf{t}_1$. This compression is an optimization for performance, not security. The low-order bits of $\mathbf{t}$ can be reconstructed from a small number of signatures and, therefore, need not be regarded as secret.

The public key $\mathbf{pk}$ is a byte encoding of the public random seed ρ and the compressed polynomial vector $\mathbf{t}_1$. The private key $\mathbf{sk}$ is a byte encoding of the master seed $\mathbf{seed}$, the secret polynomial vector $\mathbf{s}$, the noise $\mathbf{e}$, and $\mathbf{t}_0$ which contains the “lost” bits of $\mathbf{t}$ caused by compression. We note that we include the master seed instead of only the public seed into the private key, so that the private key holder can verify that $\mathbf{s}, \mathbf{e}$ are correct secret key components generated by the algorithm **KeyGen**. This can avoid certain kleptographic attacks.

Description: Generate public/secret key pair $(\mathbf{pk}, \mathbf{sk})$ via compact parameter selection.

Input: $\mathbf{seed} \leftarrow \{0, 1\}^n$

1. $(\mathbf{pk.seed}, \mathbf{sk.seed}) := (\rho, \sigma) := G(\mathbf{seed});$
 $\triangleright$ Derive ρ ($\mathbf{pk.seed}$) and σ ($\mathbf{sk.seed}$) via hash function G .
2. $\hat{\mathbf{A}} := \text{ExpandA}(\mathbf{pk.seed}) \in \mathcal{R}_q^{k \times l};$
 $\triangleright$ Construct matrix in NTT domain using ρ
3. $(\mathbf{s}, \mathbf{e}) := \text{Sample}(S_{\eta_s}^l \times S_{\eta_e}^k, \mathbf{sk.seed}) \in \mathcal{R}^l \times \mathcal{R}^k;$
 $\triangleright$ Sample secret $\mathbf{s}$ and noisy $\mathbf{e}$ using σ .
4. $\mathbf{t} := \text{INTT}(\hat{\mathbf{A}} \odot \text{NTT}(\mathbf{s})) + \mathbf{e} \bmod q;$
5. $\triangleright$ Transform private key to NTT domain, then multiply in NTT domain and transform back. $\triangleleft$
6. $\mathbf{t}_1 := \text{Comp}_d(\mathbf{t}); \mathbf{t}_0 := \mathbf{t} - \text{Decomp}_d(\mathbf{t}_1); \triangleright$ Compress public key
7. $\mathbf{pk} := (\mathbf{pk.seed}, \mathbf{t}_1);$
8. $\mathbf{sk} := (\mathbf{seed}, \mathbf{s}, \mathbf{e}, \mathbf{t}_0);$

Output: public/secret key pair $(\mathbf{pk}, \mathbf{sk})$

Algorithm 1: KeyGen

2.3.2 Signing

The algorithm **COMPASS-SIG.Sign** generates a digital signature encoded as a byte string for a message m . It takes as input the public key $\mathbf{pk} = (\mathbf{pk.seed}, \mathbf{t}_1)$, the secret key $\mathbf{sk} = (\mathbf{seed}, \mathbf{s}, \mathbf{e}, \mathbf{t}_0)$, and the message m . The algorithm outputs a signature $\sigma = (\mathbf{z}, c)$.

The signing algorithm proceeds as follows. **COMPASS-SIG.Sign** first expands the public seed $\mathbf{pk.seed}$ using the deterministic function **ExpandA** to obtain the matrix $\hat{\mathbf{A}} \in \mathcal{R}_q^{k \times l}$. This matrix is part of the public parameter and is reconstructed from the seed stored in the public key. Then, it computes $\mu = H(\mathbf{pk} \parallel m)$, where H is a cryptographic hash function. This binds the public key and the message together, providing message-dependent randomness via later steps.

After that, **COMPASS-SIG.Sign** generates a random vector $\mathbf{y} \in \mathcal{R}^l$ by sampling from the distribution $\tilde{S}_{\gamma_1}$ using the function **Sample**. The parameter γ_1 determines the bound on the infinity norm of $\mathbf{y}$. Using the recovered matrix $\mathbf{A} = \hat{\mathbf{A}}$, it computes the product $\mathbf{w} = \mathbf{A}\mathbf{y}$ from NTT operations $\text{INTT}(\hat{\mathbf{A}} \odot \text{NTT}(\mathbf{y}))$. We note that this is the only place we

use NTT operations, since the multiplication of small polynomials can be done efficiently without NTT.

The function $\text{Split}(\mathbf{w}, \gamma_2)$ is called to decompose $\mathbf{w}$ into a high part $\mathbf{w}_1$ and a low part $\mathbf{w}_0$, based on the bound γ_2 . This splitting is typical in lattice-based signatures to manage error terms.

The value $c = H(\mu \parallel \mathbf{w}_1)$ is computed, i.e., the hash of the concatenation of μ and the high part $\mathbf{w}_1$. Then $c' = \text{SampleInBall}_\tau(c)$ converts this hash into a vector c' (or polynomial) of Hamming weight τ (i.e., exactly τ non-zero coefficients, typically ± 1).

After computing the initial candidate values, the algorithm proceeds to the core rejection and consistency checks that determine whether the signature will be accepted or rejected. First, it computes the masked secret contribution $\mathbf{z} = \mathbf{y} + c'\mathbf{s}$ and splits $\mathbf{w} - c'\mathbf{e}$ using $\text{Split}(\cdot, \gamma_2)$ into a high part $\mathbf{u}_1$ and a low part $\mathbf{u}_0$. The vector $\mathbf{z}$ essentially hides the secret $\mathbf{s}$ by adding the random $\mathbf{y}$, while the decomposition of $\mathbf{w} - c'\mathbf{e}$ prepares an intermediate representation needed for verification.

The algorithm then applies the first rejection condition: it checks two infinity-norm bounds. If $\|\mathbf{z}\|_\infty \geq \gamma_1 - \eta_s \cdot \tau$ or $\|\mathbf{u}_0\|_\infty \geq \gamma_2 - \eta_e \cdot \tau$, the candidate is considered unsafe or likely to leak information about the secret key, and the algorithm immediately jumps back to Line#3 to sample a fresh $\mathbf{y}$ and restart the entire process. If those bounds are satisfied, a second rejection condition is evaluated to ensure high-bit consistency. The algorithm computes another split: $(\mathbf{w}'_1, \mathbf{w}'_0) = \text{Split}(\mathbf{w} - c'\mathbf{e} + c'\mathbf{t}_0, \gamma_2)$. It then checks whether $\mathbf{w}'_1$ differs from $\mathbf{u}_1$. Any mismatch indicates that the high bits of the reconstructed value do not agree with the high bits from the earlier split; this would cause verification to fail, so the algorithm again goes back to Line#3 to restart.

Next, a third rejection condition is applied to further avoid signature forgery. The algorithm tests whether $\|\delta \cdot \mathbf{z}\|^2 + \|\mathbf{w}'_0\|^2 < B \cdot \gamma_2^2$, where δ and B are global parameters. If the inequality holds, the algorithm interprets this as an undesirable (i.e., too small) norm that could weaken security or break the verification equation, so it once more restarts from Line#3.

Only when all three rejection checks are passed – meaning the norms are within allowed ranges, the high bits are consistent, and the Euclidean norm condition is not triggered – does the algorithm finalize the signature. It outputs $\sigma = (\mathbf{z}, c)$ as the final signature for the message m . In practice, the entire procedure may be repeated many times until a valid signature is found, ensuring both correctness and resistance against side-channel or leakage attacks.

In summary, the algorithm follows a “commit–challenge–response” pattern common in lattice-based Fiat–Shamir signatures. It repeatedly samples randomness, builds a candidate signature, and applies several rejection tests to ensure that the resulting signature is both valid and does not leak information about the secret key. The use of splitting and rejection conditions on the low parts is crucial for achieving a tight security proof and efficient parameters.

Input: $\text{pk} = (\text{pk.seed}, \mathbf{t}_1)$; $\text{sk} = (\text{seed}, \mathbf{s}, \mathbf{e}, \mathbf{t}_0)$; message m ; random seed ρ .

1. $\hat{\mathbf{A}} \leftarrow \text{ExpandA}(\text{pk.seed}) \in R_q^{k \times l}$.
 $\triangleright$ Recover $\hat{\mathbf{A}}$ from pk.seed .
 2. $\mu = H(\text{pk} \| m)$.
 3. $(\mathbf{y}, \rho) := \text{Sample}(\tilde{S}_{\gamma_1}^l \times \{0, 1\}^n, \rho)$; $\triangleright$ The random seed ρ is updated.
 4. $\mathbf{w} := \text{INTT}(\hat{\mathbf{A}} \odot \text{NTT}(\mathbf{y})) \in R_q^k$.
 5. $(\mathbf{w}_1, \mathbf{w}_0) := \text{Split}(\mathbf{w}, \gamma_2)$;
 6. $c = H(\mu \| \mathbf{w}_1)$; $c' = \text{SampleInBall}_\tau(c)$.
 7. $\mathbf{z} := \mathbf{y} + c' \mathbf{s}$; $(\mathbf{u}_1, \mathbf{u}_0) := \text{Split}(\mathbf{w} - c' \mathbf{e}, \gamma_2)$;
 8. **If** $\|\mathbf{z}\|_\infty \geq \gamma_1 - \eta_s \cdot \tau$ **Or** $\|\mathbf{u}_0\|_\infty \geq \gamma_2 - \eta_e \cdot \tau$:
 9. **Goto Line #3.**
 10. **End If**
 11. $(\mathbf{w}'_1, \mathbf{w}'_0) := \text{Split}(\mathbf{w} - c' \mathbf{e} + c' \mathbf{t}_0, \gamma_2)$;
 12. **If** $\mathbf{w}'_1 \neq \mathbf{u}_1$:
 13. **Goto Line #3.**
 14. **End If**
 15. **If** $\|\delta \cdot \mathbf{z}\|^2 + \|\mathbf{w}'_0\|^2 < B \cdot \gamma_2^2$:
 16. **Goto Line #3.**
 17. **End If**
- Output** Signature $\sigma = (\mathbf{z}, c)$.

Algorithm 2: $\text{Sign}(\text{pk}, \text{sk}, m)$

2.3.3 Verifying

The verification algorithm $\text{COMPASS-SIG.Verify}$ takes as input a public key $\text{pk} = (\text{pk.seed}, \mathbf{t}_1)$, an n -bit message m , and a candidate signature σ . Its purpose is to decide whether σ is a valid signature for m under the given public key.

The algorithm begins by deterministically expanding the seed pk.seed using the function ExpandA to reconstruct the matrix $\hat{\mathbf{A}} \in R_q^{k \times l}$, exactly the same matrix used during signature generation. It then computes the hash value $\mu = H(\text{pk} \| m)$, where H is a cryptographic hash function; this binds the public key and the message together for the rest of the verification. From the signature σ , which is assumed to be of the form $(\mathbf{z}, c)$, the algorithm extracts the challenge hash c and converts it into a sparse vector (or polynomial) c' of Hamming weight τ by calling $\text{SampleInBall}_\tau(c)$. This step mirrors the transformation performed inside the signing algorithm, ensuring that the same challenge representation is used.

The core verification step reconstructs a certain high-part vector that should match the one originally committed to by the signer. The algorithm computes the product $\mathbf{A}\mathbf{z}$ (where $\mathbf{A}$ is the expanded matrix) and subtracts from it $c' \cdot \text{Decomp}_d(\mathbf{t}_1)$. Here Decomp_d is a decompression function that expands the compressed public key component $\mathbf{t}_1$ into a full-size polynomial, typically $\mathbf{t}_1 \cdot 2^d + 2^{d-1}$, so that $c' \cdot \text{Decomp}_d(\mathbf{t}_1) = c' \cdot (\mathbf{t}_1 \cdot 2^d + 2^{d-1})$. The resulting vector is then split into a high part $\mathbf{w}'_1$ and a low part $\mathbf{w}'_0$ using the Split function with the same bound γ_2 that was used during signing. In a correctly generated signature, $\mathbf{w}'_1$ should equal the high part of $\mathbf{A}\mathbf{y}$ that the signer used to derive the challenge c .

After obtaining $(\mathbf{w}'_1, \mathbf{w}'_0)$, the algorithm evaluates three conditions simultaneously. First, it checks that the infinity norm of $\mathbf{z}$ is strictly smaller than $\gamma_1 - \eta_s \cdot \tau$, i.e., $\|\mathbf{z}\|_\infty < \gamma_1 - \eta_s \cdot \tau$. This is the opposite of the first rejection condition in the signing procedure

and guarantees that $\mathbf{z}$ is not too large, thereby preventing leakage of secret information. Second, it verifies that $\|\delta \cdot \mathbf{z}\|^2 + \|\mathbf{w}'_0\|^2 \geq B \cdot \gamma_2^2$, which is the opposite of the third rejection condition; this inequality ensures that the signature satisfies the required quality bound for correctness. Third, it recomputes the hash of μ concatenated with the recovered high part $\mathbf{w}'_1$ and checks whether the result equals the challenge c from the signature, i.e., $H(\mu \parallel \mathbf{w}'_1) = c$. Only if all three conditions hold simultaneously – the norm bound on $\mathbf{z}$, the quadratic bound on $(\mathbf{z}, \mathbf{w}'_0)$, and the hash match – does the algorithm output “Pass”. Otherwise it outputs “Fail”, indicating that the signature is invalid.

Input: $\text{pk} = (\text{pk.seed}, \mathbf{t}_1)$; $m \in \{0, 1\}^*$; Signature σ .

1. $\hat{\mathbf{A}} \leftarrow \text{ExpandA}(\text{pk.seed}) \in R_q^{k \times l}$;
 $\triangleright$ Recover $\hat{\mathbf{A}}$ from pk.seed .
2. $\mu = H(\text{pk} \parallel m)$;
3. $c' = \text{SampleInBall}_\tau(c)$;
 $(\mathbf{w}'_1, \mathbf{w}'_0) = \text{Split}(\text{INTT}(\hat{\mathbf{A}} \odot \text{NTT}(\mathbf{z})) - c' \cdot \text{Decomp}_d(\mathbf{t}_1))$;
4. **If** $\|\mathbf{z}\|_\infty < \gamma_1 - \eta_s \cdot \tau$ **And** $\|\delta \cdot \mathbf{z}\|^2 + \|\mathbf{w}'_0\|^2 \geq B \cdot \gamma_2^2$ **And** $H(\mu \parallel \mathbf{w}'_1) = c$:
5. **Output Pass**.
6. **End If**

Output Fail.

Algorithm 3: $\text{Verify}(\text{pk}, m, \sigma)$

3. Design Rationale

3.1 Security Foundation

The theoretical security of this scheme is based on hardness assumptions from module lattices, including MLWE, MSIS and a newly proposed hardness assumption SelfTargetMSIS^R .

Definition 1 (MLWE). *For integers k, l, q, n and real numbers η_s, η_e , we say that the advantage of $\mathcal{A}$ in solving the MLWE problem over the ring $\mathcal{R}_q := \mathbb{Z}_q[X]/(X^n + 1)$ is*

$$\text{Adv}_{k, l, \eta_s, \eta_e, q, n}^{\text{MLWE}}(\mathcal{A}) := \left| \Pr[\mathcal{A}(\mathbf{A}, \mathbf{t}) = 1 \mid \mathbf{A} \xleftarrow{\$} \mathcal{R}_q^{k \times l}; \mathbf{t} \xleftarrow{\$} \mathcal{R}_q^k] - \Pr[\mathcal{A}(\mathbf{A}, \mathbf{A}\mathbf{s} + \mathbf{e}) = 1 \mid \mathbf{A} \xleftarrow{\$} \mathcal{R}_q^{k \times l}; \mathbf{s} \xleftarrow{\$} [\eta_s]^{nl}; \mathbf{e} \leftarrow [\eta_e]^{nk}] \right|.$$

Definition 2 (MSIS). *For integers k, l, q, n and bounds γ, γ' , we say that the advantage of $\mathcal{A}$ in solving the MSIS problem over the ring $\mathcal{R}_q := \mathbb{Z}_q[X]/(X^n + 1)$ is*

$$\text{Adv}_{k, l, \gamma, \gamma', q, n}^{\text{MSIS}}(\mathcal{A}) := \Pr \left[\mathcal{A}(\mathbf{A}) = (\mathbf{z}, \mathbf{u}) \wedge \mathbf{A}\mathbf{z} = \mathbf{u} \bmod q \wedge \|\mathbf{z}\|_\infty \leq \gamma \wedge \|\mathbf{u}\|_\infty \leq \gamma' \mid \mathbf{A} \xleftarrow{\$} \mathcal{R}_q^{k \times l} \right].$$

Definition 3 (SelfTargetMSIS^R). *Suppose that $H : \{0, 1\}^* \mapsto B_\tau$ is a cryptographic hash function. For integers k, l, q, n, τ and bounds γ, γ' , we say that the advantage of $\mathcal{A}$ in solving the SelfTargetMSIS^R problem over the ring $\mathcal{R}_q := \mathbb{Z}_q[X]/(X^n + 1)$ is*

$$\text{Adv}_{k, l, \gamma, \gamma', q, n, \tau}^{\text{SelfTargetMSIS}^R}(\mathcal{A}) := \Pr \left[\begin{array}{l} \mathcal{A}(\mathbf{A}, \mathbf{t}) = (\mathbf{z}, \mathbf{u}, c, \mu) \wedge \|\mathbf{z}\|_\infty \leq \gamma \wedge \|\mathbf{u}\|_\infty \leq \gamma' \\ \wedge H(\mu \parallel (\mathbf{A}\mathbf{z} + \mathbf{u} - c' \cdot \text{Decomp}_d(\text{Comp}_d(\mathbf{t})))) = c \end{array} \mid \begin{array}{l} \mathbf{A} \xleftarrow{\$} \mathcal{R}_q^{k \times l}; \\ \mathbf{t} \xleftarrow{\$} \mathcal{R}_q^k \end{array} \right],$$

where $c' = \text{SampleInBall}_\tau(c)$.

In [KLS18], a non-standard hardness assumption named **SelfTargetMSIS** is proposed and used in the security proof of ML-DSA. The authors showed that there exists a reduction from **MSIS** to **SelfTargetMSIS** under classical random oracle model. Compared with **SelfTargetMSIS**, our newly defined hardness assumption only changes $\mathbf{t}$ into $\text{Decomp}_d(\text{Comp}_d(\mathbf{t}))$. But since the randomness of $\mathbf{t}$ does not take part in the reduction, our new problem **SelfTargetMSIS^R** allows a same reduction from **MSIS**, and this is why we claim that this new hardness assumption is as solid as **SelfTargetMSIS** from the view of provable security.

3.2 Security Model

We recall the (S)UF-CMA security of a signature scheme.

Definition 4 (UF-CMA). *For a signature scheme $\text{SIG} = (\text{KeyGen}, \text{Sign}, \text{Verify})$, the advantage of $\mathcal{A}$ is defined as*

$$\text{Adv}_{\text{SIG}}^{\text{UF-CMA}}(\mathcal{A}) := \Pr[\text{Exp}_{\text{SIG}}^{\text{UF-CMA}}(\mathcal{A}) = 1],$$

where $\text{Exp}_{\text{SIG}}^{\text{UF-CMA}}(\mathcal{A})$ is defined in Fig. 2. SIG is said to be UF-CMA if for any PPT $\mathcal{A}$ it holds that $\text{Adv}_{\text{SIG}}^{\text{UF-CMA}}(\mathcal{A}) \leq \text{negl}(\lambda)$.

$\text{Exp}_{\text{SIG}}^{\text{UF-CMA}}(\mathcal{A})$	$\mathcal{O}_{\text{Sign}}(\text{sk}, \mu)$
• $Q \leftarrow \emptyset$ • $(\text{pk}, \text{sk}) \xleftarrow{\\$} \text{KeyGen}(1^\lambda)$ • $(\mu^*, \text{sig}^*) \xleftarrow{\\$} \mathcal{A}^{\mathcal{O}_{\text{Sign}}(\text{sk}, \cdot)}(\text{pk})$ • Return $\mu^* \notin Q \wedge \text{Verify}(\text{pk}, \mu^*, \text{sig}^*) = 1$	• $\text{sig} \xleftarrow{\\$} \text{Sign}(\text{sk}, \mu)$ • $Q \leftarrow Q \cup \{\mu\}$ • Return sig

Figure 2: UF-CMA of Signature

Definition 5 (SUF-CMA). *For a signature scheme $\text{SIG} = (\text{KeyGen}, \text{Sign}, \text{Verify})$, the advantage of $\mathcal{A}$ is defined as*

$$\text{Adv}_{\text{SIG}}^{\text{SUF-CMA}}(\mathcal{A}) := \Pr[\text{Exp}_{\text{SIG}}^{\text{SUF-CMA}}(\mathcal{A}) = 1],$$

where $\text{Exp}_{\text{SIG}}^{\text{SUF-CMA}}(\mathcal{A})$ is defined in Fig. 3. SIG is said to be SUF-CMA if for any PPT $\mathcal{A}$ it holds that $\text{Adv}_{\text{SIG}}^{\text{SUF-CMA}}(\mathcal{A}) \leq \text{negl}(\lambda)$.

$\text{Exp}_{\text{SIG}}^{\text{SUF-CMA}}(\mathcal{A})$	$\mathcal{O}_{\text{Sign}}(\text{sk}, \mu)$
• $Q \leftarrow \emptyset$ • $(\text{pk}, \text{sk}) \xleftarrow{\\$} \text{KeyGen}(1^\lambda)$ • $(\mu^*, \text{sig}^*) \xleftarrow{\\$} \mathcal{A}^{\mathcal{O}_{\text{Sign}}(\text{sk}, \cdot)}(\text{pk})$ • Return $(\mu^*, \text{sig}^*) \notin Q \wedge \text{Verify}(\text{pk}, \mu^*, \text{sig}^*) = 1$	• $\text{sig} \xleftarrow{\\$} \text{Sign}(\text{sk}, \mu)$ • $Q \leftarrow Q \cup \{\mu, \text{sig}\}$ • Return sig

Figure 3: SUF-CMA of Signature

3.3 Parameter Sets

COMPASS-SIG is a lattice-based digital signature scheme whose security relies on the hardness of the Module Learning with Errors (MLWE) problem, Module Short Integer Solution (MSIS) problem and its variants. To accommodate different security requirements, we provide four parameter sets targeting 128, 256, 384, and 512 bits of security, respectively. All parameter sets use a prime modulus q , a ring degree n of either 256 or 512, and appropriate matrix dimensions ($k \times l$). For each set, the coefficients of the secret vector $\mathbf{s}$ and the error vector $\mathbf{e}$ are taken from $\{-1, 0, 1\}$ (i.e., $\eta_s = \eta_e = 1$), which improves implementation efficiency and helps control the expansion factor. The parameter d defines the number of bits used for compression and decompression, reducing the public key size. The value γ_1 is the infinity-norm bound for the random vector $\mathbf{y}$, while γ_2 serves as the splitting bound and directly determines the range of low bits (here γ_2 is given strictly by $(q - 1)/\{4, 8, 16\}$ depending on the security level). The rescaling factor δ is fixed to 4 under all security levels. The Hamming weight τ of the challenge vector increases from 30 to 120 as the security level rises, ensuring sufficient difficulty against brute-force attacks. The parameter B is used in the signature compactness check, balancing efficiency and security by setting a lower bound on $\|\delta \cdot \mathbf{z}\|^2 + \|\mathbf{w}'_0\|^2$. All parameters have been carefully chosen to guarantee moderate signature sizes, correct verification, and resistance against known lattice attacks (e.g., primal and dual attacks on LWE and restricted-CVP solver on ISIS). The exact numerical values are listed in the following table.

Table 1: COMPASS-SIG Parameter Sets

Security (bits)	q	n	k	l	η_s	η_e	d	γ_1	γ_2	τ	B
128	2081281	256	3	4	1	1	4	2^{15}	130080	30	560
256	2081281	256	7	7	1	1	5	2^{17}	520320	60	1120
384	8380417	512	5	6	1	1	5	2^{18}	1047552	78	1760
512	8380417	512	7	7	1	1	5	2^{19}	2095104	120	2240

We give the key and signature sizes for all security levels, along with the comparison with ML-DSA under similar security levels. We can see that although our scheme has larger public key sizes, the signature sizes are much smaller.

Table 2: Key and Signature Sizes for COMPASS-SIG (bytes)

Algorithm Name	Security bits	Public key	Private key	Signature
COMPASS-SIG-128	128	1664	960	2080
ML-DSA-44	128	1312	2560	2420
COMPASS-SIG-256	256	3616	2144	4080
ML-DSA-87	256	2592	4896	4627
COMPASS-SIG-384	384	5792	3136	7344
COMPASS-SIG-512	512	7648	4608	9024

Moreover, we can see that the sizes of $\text{pk} + 2 * \text{sig}$ in ML-DSA are 6152 and 11846 bytes for 128 and 256 bits security, while in our scheme, the sizes are 5824 and 11776 bytes respectively. So even a certificate is included in the message, our scheme still has less communication cost compared with ML-DSA.

3.4 Failure Analysis and Number of Repetitions

Next, we simply discuss the number of repetitions, i.e. the rejection probability in each iteration of the signing algorithm. We note that a high number of repetitions does not cause a failure, only increasing the time cost of signing algorithm.

For the first rejection condition (Line 8), we follow the discussion in ML-DSA, that the rejection probability is about $1 - \exp(-nl\tau\eta_s/\gamma_1 - nk\tau\eta_e/\gamma_2)$, which is about 60% $\sim$ 65%.

For the second rejection condition (Line 12), we only need to calculate the probability that $\mathbf{w} - c'\mathbf{e}$ and $\mathbf{w} - c'\mathbf{e} - c'\mathbf{t}_0$ disagree on high-order bits, which means that $\|\mathbf{u}_0 - c'\mathbf{t}_0\|_\infty \geq \gamma_2$, where $\mathbf{u}_0$ has its coefficients uniform in $[-\gamma_2 + \beta + 1, \gamma_2 - \beta - 1]$, and each coefficient in $c'\mathbf{t}_0$ is a sum of τ independent values uniform in $[-2^{d-1}, 2^{d-1} - 1]$. Since we lack an analytical expression for such condition, we use experiments to find the appropriate values d such that the additional rejection rates are $\approx 5\%$.

The rejection probability of the third condition (Line 15) is related to B , by a careful choice of B , we can let the rejection probability at this step to be close to 0.

Experimental results shows that the average numbers of repetitions are 3.09, 2.88, 3.25, 2.99 for different parameter sets.

Next, we show that all failing signatures are rejected in the signing procedure, thus the verification always passes for valid signatures.

Theorem 1. *The verification algorithm has perfect correctness.*

Proof. The verification fails only if one of the three condition checks fails.

For the first condition, i.e. $\|\mathbf{z}\|_\infty < \gamma_1 - \eta_s \cdot \tau$, the same condition is checked on Line 8 of the signing algorithm. Similarly, the second condition is checked on Line 15 of the signing algorithm.

For the third condition, we have that:

$$\mathbf{Az} - c' \cdot \text{Decomp}_d(\mathbf{t}_1) = \mathbf{A}(\mathbf{y} + c'\mathbf{s}) - c'(\mathbf{t} - \mathbf{t}_0) = \mathbf{Ay} + c'(\mathbf{As} - \mathbf{t}) + c'\mathbf{t}_0 = \mathbf{w} - c'\mathbf{e} + c'\mathbf{t}_0.$$

So the values $\mathbf{w}'_1$ in both the signing and the verifying algorithms are the same, which means that $\mathbf{w}'_1 = \mathbf{u}_1$. Moreover, since $\|\mathbf{u}_0\|_\infty < \gamma_2 - \eta_e \cdot \tau$, from Lemma 1, we have that $\mathbf{u}_1 = \mathbf{w}_1$. Thus $H(\mu\|\mathbf{w}'_1) = H(\mu\|\mathbf{w}_1) = c$ always holds.

We can see that the signing algorithm outputs a signature only when the signature passes the verification procedure, thus the verification algorithm always passes if it receives a valid signature. $\square$

3.5 Dissimilarity Claim

Our algorithm closely follows the standard Fiat-Shamir with abort paradigm under uniform distributions, which means that the algorithm shares a similar structure with other algorithms using the same paradigm, typically the NIST standard ML-DSA.

However, the basic design rationale of our algorithm is much different from ML-DSA, which also leads to differences in construction details. The ultimate goal of this algorithm is to offer tight security estimation and avoid unnecessary redundancies. To achieve this goal, we do not use the compress-and-hint technique in ML-DSA. Instead, we add an additional rejection condition on the Euclidean norm of the signature, enhancing the security against signature forgery.

The security analysis of our algorithm shows more differences compared to ML-DSA. In the theoretical analysis, we introduce new hardness assumptions and new methods for

bounding the min-entropy. In the concrete analysis, we build our security estimation on a new type of lattice hard problems and newly proposed lattice solving algorithms. These allow us to choose smaller parameters to achieve the same security levels, thus leads to a significant reduction in the signature size.

4. Security Claim and Analysis

The security analysis of our scheme can mostly inherit from ML-DSA. In fact, since we omit the usage of hints in ML-DSA, the security analysis of our scheme is significantly simpler. Another thing that differs our scheme from ML-DSA is the additional bound on Euclidean norm, as shown in Line 16 of the signing algorithm. However, an additional restriction on the signature only enhances the security, since we only need to add an additional condition on its basic hard problem, i.e. SelfTargetMSIS^R , that only enhances the hardness of the problem. Below, in the theoretical security analysis, we simply ignore this condition. However, in the concrete security analysis, we shall give more details about the impact of this additional restriction on the concrete security level of the signature.

4.1 Theoretical Security

Theorem 2. *Let $k, l, q, n, \tau, \eta_s, \eta_e, \gamma_1, \gamma_2$ defined as in the algorithm, $\gamma := \gamma_1 - \eta_s \cdot \tau$, $\gamma' := \gamma_2 - \eta_e \cdot \tau$.*

Let δ, ϵ satisfy the following condition: the min-entropy of $\mathbf{w}_1$ is at least $-\log \epsilon$ for probability $1 - \delta$, and p is the non-aborting probability in each iteration of the signing algorithm.

Then for any adversary $\mathcal{A}$ that against UF-CMA security of COMPASS-SIG, there exists an adversary $\mathcal{B}_1$ against MLWE problem, an adversary $\mathcal{B}_2$ against SelfTargetMSIS^R problem, and an adversary $\mathcal{B}_3$ against MSIS problem, such that

$$\text{Adv}_{\text{COMPASS-SIG}}^{\text{UF-CMA}}(\mathcal{A}) \leq L + \text{Adv}_{k,l,\eta_s,\eta_e,q,n}^{\text{MLWE}}(\mathcal{B}_1) + \text{Adv}_{k,l,\gamma,\gamma',q,n,\tau}^{\text{SelfTargetMSIS}^R}(\mathcal{B}_2).$$

and

$$\text{Adv}_{\text{COMPASS-SIG}}^{\text{SUF-CMA}}(\mathcal{A}) \leq L + \text{Adv}_{k,l,\eta_s,\eta_e,q,n}^{\text{MLWE}}(\mathcal{B}_1) + \text{Adv}_{k,l,\gamma,\gamma',q,n,\tau}^{\text{SelfTargetMSIS}^R}(\mathcal{B}_2) + \text{Adv}_{k,l,2\gamma,2\gamma',q,n}^{\text{MSIS}}(\mathcal{B}_3).$$

where

$$L = Q_S \epsilon \cdot \left(\frac{Q_S + 1}{2(1-p)^2} + \frac{Q_H}{1-p} \right) + \frac{Q_S Q_H \epsilon}{1-p} + \delta$$

under classical random oracle model, and

$$L = \frac{2Q_S \sqrt{\epsilon}}{1-p} \cdot \sqrt{Q_H + \frac{Q_S}{1-p}} + 2Q_H \sqrt{\frac{Q_S \epsilon}{1-p}} + \delta$$

here Q_S, Q_H are the numbers of queries to the signing oracle and random oracle, respectively.

Proof. This proof closely follows the existing work [BBD⁺23], and we omit the details. $\square$

Next, we show how to bound the values δ, ϵ . Since the original calculation for min-entropy in [KLS18] and [BBD⁺23] is far from tight, we bound the min-entropy of $\mathbf{w}_1$ using the Leftover Hash Lemma.

Since in all parameters we have $l \geq k$, we can see that $\{H_{\mathbf{A}}(\mathbf{y}) = \mathbf{w} := \mathbf{A}\mathbf{y} : \mathbf{A} \in R_q^{k \times l}\}$ is a universal hash family. It directly follows that $H'_{\mathbf{A}}(\mathbf{y}) = \mathbf{w}_1$ is also universal, with a min-entropy $-nk \log \frac{2\gamma_2+1}{q}$, and the min-entropy of $\mathbf{y}$ is $nl \log(2\gamma_1 - 2\eta_s\tau)$. From Leftover Hash Lemma, we can find proper value κ (we simply fix $\kappa = 0.5$), such that

$$\epsilon = (1 + \kappa) \left(\frac{2\gamma_2 + 1}{q} \right)^{nk},$$

and

$$\delta = \kappa^{-2} \cdot (2\gamma_1 - 2\eta_s\tau)^{-nl} \cdot \left(\frac{2\gamma_2 + 1}{q} \right)^{-nk}.$$

Next, we give the choices of δ, ϵ for minimizing the value of L , the additional reduction loss, under both security levels. For simplicity reason, we fix $Q_S = 2^{80}$ and $Q_H = 2^\lambda$, λ is the security parameter. The result is shown in the table below, here L_c stands for the reduction loss under classical random oracle model, and L_q stands for the reduction loss under quantum random oracle model.

Table 3: Additional Reduction Loss in Security Proof

Security Level	δ	ϵ	L_c	L_q
128	2^{-14000}	2^{-2300}	2^{-2090}	2^{-980}
256	2^{-30000}	2^{-1790}	2^{-1452}	2^{-597}
384	2^{-53000}	2^{-5118}	2^{-4652}	2^{-2133}
512	2^{-68000}	2^{-3582}	2^{-2988}	2^{-1237}

From Table 3, we can see that the additional reduction loss can be ignored under both classical and quantum models.

4.2 Concrete Security

4.2.1 Concrete Security for Key Recovery

Any MLWE instance can be converted into an LWE instance of size $n \cdot k \times n \cdot \ell$, i.e., with a secret vector of length $n_{\text{LWE}} = n \cdot \ell$ and $m_{\text{LWE}} = n \cdot k$ samples. We abuse the notations in this section such that $(\mathbf{A}, \mathbf{t})$ serves as an LWE sample instead of an MLWE sample.

Primal Attack. Primal Attack [AGVW17] reduces the LWE problem to the Unique Shortest Vector Problem (uSVP) using Kannan’s embedding technique [Kan83], and then employs lattice reduction algorithms to recover the target vector. Albrecht et al. [AFG13] first proposed the primal attack against the LWE problem.

For the security against key recovery, the attack can be performed by finding the LWE secret given LWE sample $(\mathbf{A}, \mathbf{t})$. Let $\mathbf{t}' = \begin{pmatrix} \mathbf{t} \\ 0 \end{pmatrix}$, the basis of the embedded lattice is constructed as follows:

$$\mathbf{B} = \begin{pmatrix} \mathbf{A}' & \mathbf{t}' \\ \mathbf{0}^T & 1 \end{pmatrix},$$

where $\mathbf{A}' = \begin{pmatrix} q\mathbf{I}_{m-n} & \mathbf{A} \\ \mathbf{O} & \mathbf{I}_n \end{pmatrix}$. In this embedded lattice $\mathbf{B}$, there exists a unique shortest lattice vector $\mathbf{v} = (\mathbf{e}, \mathbf{s}, 1)$, where $\mathbf{e}, \mathbf{s}$ are the LWE noise vector and secret vector respectively. Its norm is approximately $\|\mathbf{v}\| \approx \sigma\sqrt{m_{\text{LWE}} + n_{\text{LWE}}}$, which is shorter than the second shortest lattice vector $\lambda_2(\mathcal{L})$. Therefore, the LWE problem is reduced to the uSVP problem on the embedded lattice. Security evaluation models for evaluating uSVP on the embedded lattice include: the Core-SVP model [ADPS16], the usvp model in the Lattice Estimator, and the two-step mode LWE security evaluation model [XWW⁺24]. Here we rely on the LWE security evaluation model in two-step mode [XWW⁺24] for our security estimation under LWE primal attack. Research indicates that compared to the standalone BKZ mode [SE94, CN11], the two-step mode can solve LWE more efficiently, thereby optimizing the evaluation of LWE security.

Table 4: Concrete Security Parameters from the Two step under Classical and Quantum Parameter Settings [XWW⁺24]

Setting	Security Level	n_{LWE}	m_{LWE}	q	η_s	η_e	Memory($\log_2(\text{bit})$)	Time($\log_2(\text{gate})$)	d_{primal}	β_{BKZ}	β_{sieve}
Classical	128	1024	768	2081281	1	1	105.05	151.61	1793	428	489
Classical	256	1792	1792	2081281	1	1	193.75	276.01	3377	870	938
Classical	384	3072	2560	8380417	1	1	324.95	461.78	5633	1529	1602
Classical	512	3584	3584	8380417	1	1	386.44	549.35	6644	1840	1912
Quantum	80	1024	768	2081281	1	1	106.24	131.85	1793	425	494
Quantum	128	1792	1792	2081281	1	1	194.93	239.52	3377	867	944
Quantum	192	3072	2560	8380417	1	1	326.1	400.3	5633	1526	1607
Quantum	256	3584	3584	8380417	1	1	388	476.2	6644	1836	1920

β_{BKZ} : the largest blocksize β for BKZ preprocess.

To verify the security of the proposed scheme, we conducted concrete parameter evaluations for the LWE instances under traditional primal attack paths, such as **uSVP** and **BDD**. Experimental results demonstrate that, given the same security parameter settings, the computational overhead (**Time**) of traditional attack paths is higher than that of the **Two-step Mode** shown in Table 4. The evaluation framework based on the Two-step Mode provides a more accurate capture of security bottlenecks during lattice reduction, ensuring that the bit-security for each security level is robustly higher than the target thresholds.

We also give a comparison between our scheme and the NIST standard ML-DSA under same security levels.

Table 5: Comparison of Concrete Security Parameters between COMPASS-SIG and ML-DSA under Primal Attack.

Security Level	Algorithm Name	Time($\log_2(\text{gate})$)	Memory($\log_2(\text{bit})$)
128-bit	COMPASS-SIG-128	151.61	99.1
128-bit	ML-DSA-44	150.6	104.4
256-bit	COMPASS-SIG-256	276.01	193.75
256-bit	ML-DSA-87	277.0	194.1

The Dual Attack. A recent work [CMHST25] presents a refined version of the MATZOV-style dual attack, and showed that it can be used to reduce the security estimation of

Kyber (ML-KEM). However, their estimation relies on many unstable heuristic assumptions, and is not known to be applicable to other cryptographic schemes. For the sake of completion of the security estimation, we also propose the security estimation of our algorithm under [CMHST25], and show that the estimation results are also above the required security levels.

Table 6: Concrete Security Parameters from the latest Dual Attack under Classical and Quantum Parameter Settings [CMHST25].

Setting	Security Level	n_{LWE}	m_{LWE}	q	η_s	η_e	Time($\log_2(\text{gate})$)	d_{dual}	n_{enum}	n_{ft}	k_{ft}	β_{BKZ}	β_{sieve}
Classical	128	1024	768	2081281	1	1	143.76	898	4	103	6	394	401
Classical	256	1792	1792	2081281	1	1	256.53	1525	12	200	11	795	782
Classical	384	3072	2560	8380417	1	1	423.91	2449	21	389	17	1389	1347
Classical	512	3584	3584	8380417	1	1	519.04	2928	17	200	20	1727	1669
Quantum	80	1024	768	2081281	1	1	130.45	890	1	31	5	414	427
Quantum	128	1792	1792	2081281	1	1	225.45	1475	15	184	9	805	798
Quantum	192	3072	2560	8380417	1	1	371.88	2540	15	360	15	1404	1369
Quantum	256	3584	3584	8380417	1	1	450.45	2874	17	200	18	1727	1676

According to the evaluation results presented in Table 6 with $n_{\text{LWE}} = n \cdot l$ and $m_{\text{LWE}} = n \cdot k$, the estimated cost of the coded-dual filtering stage is significantly lower than the corresponding primal-attack estimates. However, this comparison should be interpreted with caution. The coded-dual cost model assumes that the final sub-LWE solving step is negligible, whereas in an actual key-recovery setting this step may require solving a residual LWE instance of dimension $n - n_{\text{enu}}$. If the residual instance is solved by a primal attack, its cost should not be ignored, especially when n_{enu} is small. Consequently, although the reported dual-attack estimate is substantially lower, the model may underestimate the full cost of complete secret recovery.

Hybrid Attacks. Hybrid attacks against the Learning with Errors (LWE) problem combine lattice reduction techniques (such as BKZ) with meet-in-the-middle or guess-and-verify strategies to exploit the properties of small or sparse secret vectors. These attacks generally fall into two categories:

- **Primal Hybrid Attack (bdd_hybrid):** As proposed in [ACW19], this approach partitions the secret $\mathbf{s} = (\mathbf{s}_1, \mathbf{s}_2)$, where $\mathbf{s}_2$ is guessed and $\mathbf{s}_1$ is recovered using a BDD solver on a reduced-dimension lattice. By utilizing a high-probability BDD solver and dynamic dimension selection, it balances the costs of lattice preprocessing and decoding, effectively transforming a high-dimensional problem into a more manageable lower-dimensional one.
- **Dual Hybrid Attack:** Proposed in [EJK20], this serves as an extension of the hybrid framework to the dual attack. It splits the matrix $\mathbf{A}$ and secret $\mathbf{s}$, constructing a new LWE sample with reduced dimension but increased noise e' . The attack then recovers the partial secret through recursive enumeration and a distribution distinguisher (e.g., Levy transform).

Despite their theoretical advantages in specific parameter sets (such as sparse secrets), our comprehensive security evaluations show that for the parameter settings considered in this work, these hybrid attacks do not yield lower security margins compared to the standard primal attack or dual attack.

4.2.2 Concrete Security for Signature Forgery

Instead of considering the concrete hardness of the SelfTargetMISIS^R problem, we take a similar approach from ML-DSA [SBN⁺20]: first pick a commitment vector $\mathbf{w}_1$, compute $H(\mu\|\mathbf{w}_1) = c$, then find $\mathbf{z}, \mathbf{u}$ such that $\mathbf{A}\mathbf{z} + \mathbf{u} = \mathbf{w}_1 \cdot (2\gamma_2) + c\mathbf{t}_1 \cdot 2^d$, $\|\mathbf{z}\|_\infty < \gamma_1 - \beta$, $\|\mathbf{u}\|_\infty \leq \gamma_2$. Note that by the definition of the new hard problem SelfTargetMISIS^R , the bound of $\mathbf{u}$ is simply γ_2 , without introducing the additional term $2^{d-1} \cdot \tau$ as in ML-DSA. Let $\mathbf{t}' = c\mathbf{t}_1 \cdot 2^d - \mathbf{w}_1 \cdot (2\gamma_2)$, the problem can be written as: $\mathbf{A}\mathbf{z} + \mathbf{u} = \mathbf{t}'$.

The only difference between this problem and the standard MISIS problem, is that $\mathbf{t}'$ is not a uniform random vector. However, there is currently no method for exploiting the hidden structure of $\mathbf{t}'$, and this problem can only be solved using the same method as solving the standard MISIS problem. For the hardness of this problem, we use the estimation method presented in [WVG25], that turns MISIS under infinity norm into a variant of SVP problem called restricted-SVP. We note that this is currently the only estimation for MISIS under gate-count model.

We give a brief summary for the estimation method in [WVG25]:

Definition 6. Given $\mathcal{R} : \mathbb{R}^d \mapsto \{0, 1\}$ (called restriction), $\mathcal{L}$ is a d -dimensional lattice, let $\mathcal{P}(\text{len}) = \Pr(\mathcal{R}(\mathbf{v}) = 1 | \mathbf{v} \in \mathcal{L} \wedge \|\mathbf{v}\| \leq \text{len})$, we require that $\mathcal{P}(\text{len}) \geq \mathcal{P}(\text{len}')$ for any $\text{len} < \text{len}'$. The restricted SVP ($\text{RSVP}(\mathcal{L}, \mathcal{R})$) aims to find a non-zero vector $\mathbf{v} \in \mathcal{L}$ such that $\mathcal{R}(\mathbf{v}) = 1$.

We call $\mathcal{P}(\text{len})$ the related probability function, which describes the relation between the restriction and the length of lattice vectors.

Similarly, we define the restricted CVP problem: for a target vector $\mathbf{t}$, let $\mathcal{P}_{\mathbf{t}}(\text{len}) = \Pr(\mathcal{R}(\mathbf{v}) = 1 | \mathbf{v} \in \mathcal{L} \wedge \|\mathbf{v} - \mathbf{t}\| \leq \text{len})$, we require that $\mathcal{P}(\text{len}) \geq \mathcal{P}(\text{len}')$ for any $\text{len} < \text{len}'$. The restricted CVP ($\text{RCVP}(\mathcal{L}, \mathcal{R})$) aims to find a vector $\mathbf{v} \in \mathcal{L}$ such that $\mathcal{R}(\mathbf{v}) = 1$.

For calculating the related probability function, we use the following lemma in [WVG25] that approximates a uniform distribution from a high-dimensional ball by a high-dimensional Gaussian:

Lemma 2. (1) Uniform distribution on $\mathcal{B}^d(\text{len})$ is close to a d -dimensional spherical Gaussian $\mathcal{N}\left(\mathbf{0}, \frac{\text{len}^2}{d} \cdot \mathbf{I}\right)$.

(2) The probability of $\|\mathbf{v}\|_\infty \leq B$ for $\mathbf{v}$ uniform in $\mathcal{L} \cap \mathcal{B}^d(\text{len})$ is approximately $\left(1 - 2\Phi\left(-\frac{\sqrt{d} \cdot B}{\text{len}}\right)\right)^d$, where Φ is the cumulated distribution function for the standard Gaussian $\mathcal{N}(0, 1)$.

For solving MISIS, we first construct the following lattice using the rescaling method:

$$\mathbf{B}_\delta = \begin{pmatrix} q\mathbf{I} & \mathbf{A} \\ \mathbf{0} & -\delta\mathbf{I} \end{pmatrix},$$

where $\delta \approx \frac{\gamma_2}{\gamma_1 - \beta}$, and $(\frac{\mathbf{u}}{\delta\mathbf{z}})$ is one of its short vectors. The related probability in the lattice can be defined as: $\mathcal{P}(\text{len}) = \left(1 - 2\Phi\left(-\frac{\sqrt{d} \cdot \gamma_2}{\text{len}}\right)\right)^{n(k+l)}$. Then we can use Kannan's embedding to map restricted-CVP into restricted-SVP, whose details are shown in [Kan83]. The result is given in Table 7.

Batch approximate CVP vs approximate CVP. We note that to forge a signature, instead of solving approximate CVP, it suffices for us to solve a batched version: give many different target vectors $\mathbf{t}'_1, \dots, \mathbf{t}'_n$, find a close lattice vector to any one of them.

It has been shown in many works [DLvW20, EK20, XWG25, KKNM25] such that batch-CVP might be easier to solve than CVP. However, we claim that the same thing does not happen for approximate CVP. The reason is simple: for the acceleration method used in batch-CVP, one should first use a fast polynomial-time algorithm to generate a partial solution for each CVP instance, then use an exponential-time algorithm to find the exact solution for one of these partial solutions. However, approximate CVP already has exponential number of possible solutions instead of one, so it is possible to generate enough partial solutions for a single approximate CVP, which means that batching shall not lead to a speedup.

Amplifying the Hardness of approximate-CVP $^\infty$. We note that we introduced an additional rejection condition in the signature algorithm, i.e. letting the vector $\delta\mathbf{z}\|\mathbf{u}$ have large enough Euclidean norm. Since MISIS can be solved by approximate-CVP $^\infty$, this new problem can be solved by a variant of approximate-CVP defined as follows:

Definition 7 (Shell-CVP). *For any lattice $\mathcal{L}$ and a target vector $\mathbf{t}$, Shell-CVP $_{B_\infty, B_2}$ aims to find a lattice vector $\mathbf{v} \in \mathcal{L}$ with $\|\mathbf{v} - \mathbf{t}\|_\infty \leq B_\infty$ and $\|\mathbf{v} - \mathbf{t}\|_2 > B_2$.*

Note that for the simplicity of further discussions, we give the absolute version of the definition, i.e. B_∞ and B_2 are concrete bounds instead of approximation factors related to the lengths of shortest lattice vectors. We also define its counterpart, Shell-SVP problem:

Definition 8 (Shell-SVP). *For any lattice $\mathcal{L}$, Shell-SVP $_{B_\infty, B_2}$ aims to find a lattice vector $\mathbf{v} \in \mathcal{L}$ with $\|\mathbf{v}\|_\infty \leq B_\infty$ and $\|\mathbf{v}\|_2 > B_2$.*

It directly follows that this new problem is harder than approximate-CVP $^\infty$ with bound B_∞ , as not all solutions of approximate-CVP $^\infty$ are solutions to this new problem. However, estimating the concrete hardness of this new problem turns out to be a difficult task. We first consider the hardness of Shell-SVP instead of Shell-CVP. Specifically, this problem does not satisfy the definition of restricted SVP, so the estimation for restricted SVP cannot apply. We list a few attempts that we used to solve Shell-SVP:

(1) Use polynomial-time discrete Gaussian sampling algorithms, such as Klein sampler or Peikert sampler to sample a large amount of lattice vectors under the discrete Gaussian distribution D_σ , search among them to find the target vector, as suggested in [Kan83]. We can omit the time cost of sampling, and the complexity is thus bounded by the inverse of success probability, i.e. $(1 - 2\Phi(-\frac{B_\infty}{\sigma}))^{-d}$. However, the standard deviation σ must be sufficiently large for the polynomial-time sampler to apply. A simple calculation shows that such method is inefficient.

(2) Combine BKZ with a sieving-style algorithm under the infinity norm. The main problem of this approach is that there is currently no implementation for infinity-norm lattice sieving. The only theoretical analysis shows that the complexity is about $2^{0.62d}$ compared with $2^{0.292d}$ for Euclidean norm. So this method is also inefficient.

(3) Select a proper value $B_2/\sqrt{d} < v < B_\infty$ such that the vector $\mathbf{v} = \{v, \dots, v\}$ lies in the shell, then find a lattice vector in the shell by solving restricted-CVP (and further into restricted-SVP through Kannan's embedding) with target vector $\mathbf{v}$. This turns out to be the most efficient method, and the hardness estimation of such approach can be given by the estimator in [WXG25] with simple modification. For Shell-CVP instead of Shell-SVP, we only need to change the target vector $\mathbf{v}$ into $\mathbf{v} + \mathbf{t}$, which does not change the time complexity.

The only problem we need to solve is approximating the related probability function $\mathcal{P}(\text{len})$ for the restricted-CVP problem. From Lemma 2, we can calculate $\mathcal{P}(\text{len})$ as follows:

For $\mathcal{P}(\text{len})$, we assume that a uniform random vector $\mathbf{x} = (x_1, \dots, x_d)$ in $\mathcal{B}_2(\text{len}, \mathbf{v})$ by $\mathcal{N}\left(\mathbf{v}, \frac{\text{len}^2}{d} \cdot \mathbf{I}\right)$, where each element x_i in the vector follows $\mathcal{N}\left(v, \frac{\text{len}^2}{d}\right)$. We first place the restriction $\|\mathbf{x}\|_\infty \leq B_\infty$ on $\mathbf{x}$. The probability for $\mathbf{x}$ to pass this restriction is $\left(\Phi\left(\frac{\sqrt{d} \cdot (B_\infty - v)}{\text{len}}\right) - \Phi\left(-\frac{\sqrt{d} \cdot (B_\infty + v)}{\text{len}}\right)\right)^d$. Under our parameter selection, $\Phi\left(-\frac{\sqrt{d} \cdot (B_\infty + v)}{\text{len}}\right)$ is a negligible value, so we simplify it into $\Phi\left(\frac{\sqrt{d} \cdot (B_\infty - v)}{\text{len}}\right)^d$.

Now, for $\mathbf{x}$ passing the first restriction, we can see that each x_i follows a truncated Gaussian, which is the normal distribution $\mathcal{N}\left(\mathbf{v}, \frac{\text{len}^2}{d} \cdot \mathbf{I}\right)$ under the condition $|x| \leq B_\infty$. To show the probability that $\mathbf{x}$ passes the second restriction, i.e. $\|\mathbf{x}\| > B_2$, we only need to calculate the probability that $x_1^2 + \dots + x_d^2 > B_2^2$. Since d is sufficiently large, we approximate the sum of d independent variables following the same distribution using the central limit theorem, that claims $x_1^2 + \dots + x_d^2$ approximately follows $\mathcal{N}(d\bar{\mu}, d\bar{\sigma}^2)$, where $\bar{\mu}$ and $\bar{\sigma}^2$ are the mean value and variance of the truncated distribution, respectively. We show that $\bar{\mu}$ and $\bar{\sigma}^2$ can be calculated as follows:

Suppose x follows the distribution $\mathcal{N}(\mu, \sigma)$ under condition $|x| \leq B$. To compute $\bar{\mu} = \mathbb{E}(x^2 \mid |x| \leq B)$ and $\bar{\sigma}^2 = \mathbb{E}(x^4 \mid |x| \leq B) - (\mathbb{E}(x^2 \mid |x| \leq B))^2$, we consider the random variable $y = \frac{x - \mu}{\sigma}$ following distribution $\mathcal{N}(0, 1)$ under condition $y \in [\frac{-B - \mu}{\sigma}, \frac{B - \mu}{\sigma}] := [a, b]$. Then,

$$\begin{aligned}\mathbb{E}(x^2 \mid |x| \leq B) &= \mathbb{E}((\sigma y + \mu)^2 \mid y \in [a, b]) \\ &= \sigma^2 M_2(a, b) + 2\sigma\mu M_1(a, b) + \mu^2, \\ \mathbb{E}(x^4 \mid |x| \leq B) &= \mathbb{E}((\sigma y + \mu)^4 \mid y \in [a, b]) \\ &= \sigma^4 M_4(a, b) + 4\sigma^3\mu M_3(a, b) + 6\sigma^2\mu^2 M_2(a, b) + 4\sigma\mu^3 M_1(a, b) + \mu^4,\end{aligned}$$

where $M_n(a, b)$ is the n -th raw moment of $\mathcal{N}(0, 1)$ restricted to the interval $[a, b]$ for $1 \leq n \leq 4$. From the density function $\rho(x)$ and cumulative function $\Phi(x)$ of the distribution $\mathcal{N}(0, 1)$, we get that

$$\begin{aligned}M_n(a, b) &= \mathbb{E}(x^n \mid x \leftarrow \mathcal{N}(0, 1), x \in [a, b]) = \int_{-\infty}^{\infty} x^n \frac{\rho(x)}{\Phi(b) - \Phi(a)} \mathbf{1}_{[a, b]}(x) dx \\ &= \frac{1}{\Phi(b) - \Phi(a)} \int_a^b x^n \rho(x) dx,\end{aligned}$$

where $\int_a^b x^n \rho(x) dx := c_n$ can be derived from the recurrence formula $c_n = (n - 1) \cdot c_{n-2} + a^{n-1} \rho(a) - b^{n-1} \rho(b)$, together with the initial conditions $c_0 = \Phi(b) - \Phi(a)$ and $c_1 = \rho(a) - \rho(b)$.

So we give a nice approximation for the related probability $\mathcal{P}(\text{len})$, that can be used for the security estimation for signature forgery. The results are shown in Table below, and the numbers in parentheses are the estimation results for normal MISIS $^\infty$, i.e. approximate-CVP $^\infty$ without considering the bounds on Euclidean norm.

Table 7: Security Strength of SIS problem in signature

Security Level	β_{BKZ}	Classical Time ($\log_2(\text{gate})$)	Quantum Time ($\log_2(\text{gate})$)
128	409 (366)	145.6 (133.8)	136.62 (116.7)
256	913 (832)	287.2 (264.7)	248.55 (230.5)
384	1432 (1320)	433.5 (402.2)	376.3 (349.9)
512	1898 (1758)	565.4 (526.0)	495.3 (457.2)

From Table 7, we can see that even if we consider approximate-CVP instead of Shell-CVP in the security estimation, the security against signature forgery can still satisfy the required security level, while the additional bound on Euclidean norm in Shell-CVP increases the security level by 11.8-39.4 bits.

4.2.3 Concrete Security for Strong Signature Forgery

We note that in the security proof, the additional condition for strong signature forgery is reduced into finding a short lattice vector $\mathbf{z}' \parallel \mathbf{u}'$, such that $\|\mathbf{z}'\|_\infty \leq 2(\gamma_1 - \beta)$ and $\|\mathbf{u}'\|_\infty \leq 2\gamma_2$. However, it is not hard to understand that finding such a lattice vector cannot always result in a signature forgery.

Specifically, for a strong signature forgery, we require that there exists a valid signature $\mathbf{z}$ and $\mathbf{u}$ is the low bits in $\text{Split}(\mathbf{A}\mathbf{z} - \mathbf{c}\mathbf{t}')$ such that $\mathbf{z}' - \mathbf{z}$ and $\mathbf{u}' - \mathbf{u}$ form a valid signature, i.e. $\|\mathbf{z}' - \mathbf{z}\|_\infty \leq \gamma_1 - \beta$ and $\|\mathbf{u}' - \mathbf{u}\|_\infty \leq \gamma_2$. However, for a random lattice vector satisfying only $\|\mathbf{z}'\|_\infty \leq 2(\gamma_1 - \beta)$ and $\|\mathbf{u}'\|_\infty \leq 2\gamma_2$, such condition rarely holds even if we perform 2^{80} signature queries, which is the upper bound required in the proposal.

So for a strong signature forgery to be valid, we must place additional restriction on the lattice vector $\mathbf{z}' \parallel \mathbf{u}'$, which forms a restricted SVP instead of approximate SVP suggested by the security proof. Next we calculate the restriction function, and show that the probability of a lattice vector passing the restriction is too small for the strong signature forgery to have a lower or even similar complexity comparing with normal signature forgery. Since we use restricted SVP on the same lattice as the basis algorithm for attacks on both unforgeability and strong unforgeability, it suffices for us to compare only the related probability function $\mathcal{P}(\text{len})$.

For calculating the related probability function, we use the following proposition.

Proposition 1. For $\mathbf{x} \xleftarrow{\$} \mathcal{B}_\infty^n(\mathbf{0}, a)$, the probability of a vector in $\mathcal{B}^n(\mathbf{x}, r)$ belonging to $\mathcal{B}_\infty^n(\mathbf{0}, a)$ is approximately $[1 - 2\Phi(-\frac{2a}{\sigma}) - \frac{\sigma}{a}(\rho(-\frac{2a}{\sigma}) - \rho(0))]^n$, where $\sigma = \frac{r}{\sqrt{n}}$, $\rho(x)$ and $\Phi(x)$ are the density function and cumulative function of the distribution $\mathcal{N}(0, 1)$.

Proof. From Lemma 2, it suffices to compute $\mathbb{E}[\text{Pr}_{\mathbf{x}}]$, where

$$\begin{aligned} \text{Pr}_{\mathbf{x}} &= \Pr\left(\|\mathbf{x} + \mathbf{t}\|_\infty < a \mid \mathbf{t} \leftarrow \mathcal{N}(\mathbf{0}, \frac{r^2}{n}\mathbf{I})\right) \\ &= \prod_{i=1}^n \Pr\left(x_i + t_i < a \mid t_i \leftarrow \mathcal{N}(0, \frac{r^2}{n})\right). \end{aligned}$$

Write $\text{Pr}_{x_i} = \Pr\left(x_i + t_i < a \mid t_i \leftarrow \mathcal{N}(0, \frac{r^2}{n})\right)$. Since $\mathbf{x}$ is uniform random, $\mathbb{E}[\text{Pr}_{\mathbf{x}}] = \prod_{i=1}^n \mathbb{E}[\text{Pr}_{x_i}] = \mathbb{E}[\text{Pr}_x]^n$, where $\text{Pr}_x = 1 - \Phi\left(\frac{-a+x}{\sigma}\right) - \Phi\left(\frac{-a-x}{\sigma}\right)$. From the property

$\int \Phi(x)dx = x\Phi(x) + \rho(x)$, it holds that

$$\mathbb{E}[\text{Pr}_x] = \frac{1}{2a} \int_{-a}^a \text{Pr}_x dx = 1 - 2\Phi\left(-\frac{2a}{\sigma}\right) - \frac{\sigma}{a} \left(\rho\left(-\frac{2a}{\sigma}\right) - \rho(0) \right).$$

□

From the expression above, we can calculate the related probability function under all parameters, and the result shows that the related probability for additional strong signature forgery is much smaller than that for normal signature forgery, which means that the additional requirement for strong forgery in fact has little impact on the concrete security estimation.

5. Performances

In this section, we provide a comprehensive evaluation of the performance characteristics and practical applicability of the proposed COMPASS-SIG. The assessment is structured into two primary parts. First, in Section 5.1, we present a theoretical analysis of the implementational tradeoffs. We examine the underlying computational costs of the symmetric cryptographic primitives and the rejection sampling mechanism, while emphasizing the system-level bandwidth advantages—specifically the drastic reduction in secret key and signature sizes—derived from our modified MSIS formulation. Second, in Section 5.2, we report the quantitative benchmarking results obtained from our pure ANSI C reference implementation. By providing a detailed cycle-count breakdown for the core operations across all security levels on an x86 architecture, we demonstrate that the proposed scheme not only yields significantly more compact key sizes, but also achieves substantial computational speedups in both the **Sign** and **Verify** procedures over the current ML-DSA standard.

5.1 Performances Analysis

5.1.1 Implementation Considerations and Tradeoffs

Symmetric Primitives and Rejection Sampling. In Fiat-Shamir with aborts signatures, the most compute-intensive phase is the iterative rejection sampling during the **Sign** procedure. Each iteration requires expanding the masking vector $\mathbf{y}$ via an Extendable-Output Function (XOF), executing NTT multiplications, and computing the challenge hash. Therefore, the average number of **while** loop iterations dictates the overall signing latency. By carefully tuning the Euclidean norm bound B and the infinity norm restrictions on $\mathbf{z}$, our design ensures a highly efficient acceptance rate without expanding the parameter dimensions.

Tradeoffs between Key/Signature Sizes and Bandwidth. A central design philosophy of our scheme is optimizing the parameters that are most critical for real-world network performance, even at the cost of slightly increasing less critical dimensions. As empirical data shows, our public key (PK) is marginally larger than that of ML-DSA (e.g., 1664 bytes versus 1312 bytes at the 128-bit security level). However, this deliberate tradeoff yields a massive reduction in both the secret key (SK) and signature sizes. At the 128-bit level, our secret key is reduced to merely 960 bytes compared to ML-DSA-44’s 2560 bytes, and the signature size is compressed to 2080 bytes versus 2420 bytes.

From an engineering and protocol perspective, this asymmetric size profile offers tremendous practical value. In high-frequency network communication scenarios such as Transport Layer Security (TLS) handshakes, public keys are typically static and can be efficiently cached or distributed via PKI. Conversely, the signature itself is a dynamic payload that must be computed and transmitted during every single interactive session. Therefore, minimizing the signature size directly reduces the communication bandwidth per transaction, leading to lower transmission latency and reduced packet fragmentation. The significant bandwidth savings achieved by shrinking the signature far outweigh the marginal one-time cost of storing a slightly larger public key.

5.1.2 Performance Scaling Across Security Levels

The computational overhead naturally scales with the matrix dimensions and polynomial degrees across the four security levels. However, a critical metric for signature stability—the expected number of rejection sampling iterations—remains highly consistent.

In the Fiat-Shamir with aborts paradigm, algorithm performance is highly sensitive to the rejection probability during the signing phase. Based on a statistical evaluation of 10,000 signature generations across all security levels (from 128-bit to 512-bit), the average number of `while` loop iterations remains remarkably stable, ranging narrowly between 2.88 and 3.25 iterations per signature. This consistency demonstrates that the parameter scaling perfectly preserves the algorithmic efficiency without inducing exponential rejection penalties at higher security levels.

Furthermore, a granular analysis of the rejection conditions reveals the efficacy of our improved Module Short Integer Solution (MSIS) formulation. In our `Sign` procedure, the rejection rate for the Euclidean norm check (Line 15 of the signing algorithm, checking the bound B) is virtually zero—specifically, 0.0002 at the 128-bit level and 0 at higher levels. This empirical evidence indicates that our modified Euclidean norm bound B is meticulously calibrated. It successfully tightens the mathematical security bounds against advanced lattice attacks without inadvertently increasing the computational burden of the rejection sampling loop, achieving an optimal balance between theoretical security robustness and empirical execution speed.

5.2 Reference Implementation

In this section, we present the implementation details and performance evaluation of COMPASS-SIG. To provide a comprehensive assessment, our evaluation encompasses two distinct versions: a pure ANSI C reference implementation and a platform-specific optimized implementation leveraging AVX2 instructions. The reference implementation is developed to maximize portability and algorithmic clarity, providing an unambiguous, architecture-agnostic baseline to assess the inherent computational efficiency of the algorithm. Complementing this, the AVX2-optimized version demonstrates the high-throughput performance potential on modern x86 platforms by exploiting data-level parallelism.

5.2.1 Performance Results on x86 Architecture

To evaluate the sequential execution efficiency of the core cryptographic operations, we benchmarked the C reference implementation against ML-DSA.

Benchmarking Methodology The performance profiling was conducted on a virtualized workstation powered by an AMD Ryzen 7 8745HS quad-core processor and 8 GB of RAM, running Ubuntu 22.04. The source code was compiled using the GCC compiler with the `-O3` optimization flag to maximize compiler-level efficiency. To ensure the reliability of the cycle count measurements and mitigate OS-level noise, CPU dynamic frequency scaling was disabled. The reported clock cycles for Key Generation (**KeyGen**), Signing (**Sign**), and Verification (**Verify**) were averaged over 10,000 independent executions.

Performance Results Table 8 summarizes the average clock cycle counts of our proposed algorithm across four security levels, alongside a direct comparison with the corresponding ML-DSA parameters at the 128-bit and 256-bit security levels.

Table 8: Performance Comparison of Signature Algorithms (Clock Cycles on x86 CPU)

Algorithm	Security (bits)	KeyGen	Sign	Verify
ML-DSA-44 (C Ref)	128	331,703	1,474,335	375,036
COMPASS-SIG-128 (C Ref)	128	328,537	1,096,226	295,441
COMPASS-SIG-128 (AVX2)	128	156,890	384,936	151,876
ML-DSA-87 (C Ref)	256	931,215	3,380,411	975,866
COMPASS-SIG-256 (C Ref)	256	835,796	2,514,794	804,670
COMPASS-SIG-256 (AVX2)	256	378,248	894,835	367,472
COMPASS-SIG-384 (C Ref)	384	1,094,420	3,302,735	1,168,198
COMPASS-SIG-384 (AVX2)	384	541,841	1,428,775	591,102
COMPASS-SIG-512 (C Ref)	512	1,604,747	4,538,370	1,709,494
COMPASS-SIG-512 (AVX2)	512	728,644	1,695,434	709,480

Cycle Breakdown and Algorithmic Advantage Based on the empirical data presented in Table 8, our proposed signature scheme demonstrates a profound performance advantage over ML-DSA all core operational phases.

Accelerated Signing: The most computationally expensive operation in lattice-based signatures is the **Sign** procedure, largely due to the iterative nature of rejection sampling. Our algorithm achieves a remarkable speedup of approximately $1.34\times$ at both the 128-bit and 256-bit security levels compared to ML-DSA. This significant acceleration is fundamentally driven by the optimized mathematical bounds derived from our modified Module Short Integer Solution (MSIS) formulation. By drastically reducing the dimensionality of the secret key and the size of the masking vectors, the per-iteration computational burden—specifically the NTT conversions and the symmetric XOF expansions required to generate candidate vectors—is heavily minimized. Furthermore, because our finely tuned Euclidean norm bounds maintain a stable and low rejection rate, the algorithm successfully reaps the benefits of smaller polynomial operations without suffering from an increased number of **while** loop restarts.

Streamlined Verification: The **Verify** function exhibits a $1.27\times$ and $1.21\times$ acceleration at 128-bit and 256-bit security levels respectively. Since signature verification in the Fiat-Shamir paradigm is rejection-free and deterministic, this speedup serves as a direct indicator of the reduced arithmetic complexity. Our scheme operates on significantly smaller signature sizes (e.g., a 2080-byte signature versus ML-DSA-44’s 2420-byte signature at the 128-bit level). Consequently, parsing the signature, executing the inverse

NTT operations, and reconstructing the commitment hash require fewer CPU cycles and exert lower pressure on the cache subsystem.

Ultimately, the baseline benchmarking confirms that the proposed scheme successfully leverages tighter MSIS assumptions not only to enhance theoretical security but also to achieve a strict empirical reduction in computational latency, securing a decisive performance lead over existing standards.

5.2.2 AVX2 Optimized Implementation

To complement our portable ANSI C reference implementation, we developed a highly optimized implementation targeting modern x86 processors equipped with the Advanced Vector Extensions 2 (AVX2) instruction set, as shown in Table 8. Instead of relying on generic optimization frameworks, we engineered a bespoke AVX2 architecture tailored specifically to the unique algebraic structures of our scheme. This approach fully exploits data-level parallelism to accelerate both the symmetric cryptographic primitives and the core polynomial arithmetic autonomously.

Symmetric Primitives A significant portion of the computational latency during the Sign and KeyGen procedures is consumed by the extendable-output function (XOF) used for matrix expansion and rejection sampling. To mitigate this, we implemented a highly parallelized 4-way Keccak-f[1600] permutation. By leveraging the 256-bit `_m256i` registers, our implementation evaluates four independent Keccak states simultaneously. This parallel architecture dramatically reduces the latency of generating the expanded matrix $\hat{\mathbf{A}}$ and the masking vectors $\mathbf{y}$.

Dual-Parameter Polynomial Arithmetic and NTT A distinctive engineering highlight of our AVX2 implementation is its native support for our dual-modulus architecture. To cover all four security levels, our scheme operates on two distinct parameter sets: $q = 2081281$ with $n = 256$ for Levels 1 and 2, and $q = 8380417$ with $n = 512$ for Levels 3 and 4.

Since both moduli require up to 23 bits of precision, the polynomial coefficients must be represented as 32-bit signed integers (`int32_t`). Consequently, a single 256-bit AVX2 register packs exactly 8 coefficients, achieving an $8\times$ parallelization factor for element-wise vector operations. To maximize instruction throughput without introducing branch penalties, we independently designed two separate, highly unrolled sets of AVX2 intrinsic codes tailored to each modulus:

- **For $q = 2081281, n = 256$:** We constructed a dedicated set of vectorized routines for the 8-layer Number-Theoretic Transform (NTT). This includes custom-derived Montgomery reduction multipliers and Barrett constants calibrated specifically for $q = 2081281$. The Cooley-Tukey butterfly operations leverage the `_mm256_mul_epu32` and `_mm256_srli_epi64` instructions to execute 32-bit modular arithmetic natively within the vector registers.
- **For $q = 8380417, n = 512$:** For this high-security parameter set, we engineered advanced 32-bit SIMD reduction strategies and substantially expanded the memory access patterns and shuffle instructions (such as `_mm256_permutevar8x32_epi32`) to seamlessly handle the larger $n = 512$ polynomial arrays across a 9-layer NTT. This independent memory-routing design ensures that the L1 cache bandwidth is optimally utilized over the extended memory footprint.

Rejection Sampling and Data Packing The AVX2 instruction set is critically instrumental in accelerating the rejection sampling loop—the most prominent bottleneck in Fiat-Shamir signatures. When extracting uniformly bounded coefficients or evaluating the norm conditions, we employ 32-bit vectorized comparison instructions (`_mm256_cmpgt_epi32`) and blend operations (`_mm256_blendv_epi8`). This autonomous mechanism allows our algorithm to verify the Euclidean norm B and infinity norm restrictions in a strictly branchless manner, effectively preventing pipeline stalls caused by branch mispredictions. Furthermore, the bit-packing and unpacking routines for signature serialization are fully vectorized from the ground up, replacing sequential bitwise operations with high-speed SIMD shifts and masks.

6. Feature Statement

In this section, we make a statement on the features of our algorithm.

- *Novelty.* In this proposal, we present novel techniques in both theoretical and concrete security analysis to show the redundancy in the parameter choice of ML-DSA. Specifically, we combine the rescaling method used in the concrete security analysis and a more accurate computation on the min-entropy, together to minimize the size of the signature without compromising its unforgeability. We also give a concrete analysis to show that, unlike what was indicated by the designers of ML-DSA, the additional requirement of strong unforgeability does not require a larger parameter. Removing the redundancy allows us to give a new design for lattice-based signature from Fiat-Shamir with abort that optimizes both the signature size and the time cost of all algorithms. In addition to the contribution points described above, we also present a new lattice-based hard problem, i.e. Shell-CVP that further amplifies the security against signature forgery.
- *Simplicity.* Our scheme satisfies simplicity from design and implementation. Our scheme follows the standard Fiat-Shamir with abort paradigm for lattice-based signature, whose security reduction is well studied. Specifically, compared with ML-DSA, we remove the usage of hints, which further simplifies the design. Our scheme is based on hard problems on module lattices with prime modulus which enjoy worst-case security, and most of them have well-studied hardness. To ease the concerns about the new hard problem Shell-CVP, we show that the hardness of approximate CVP without considering the Shell-CVP variant is also sufficient to achieve the security level. For implementation, we avoid the usage of heavy operations such as Gaussian sampling, and show that our scheme can be constructed from only basis operations including NTT, mod-mult and hash functions.
- *Flexibility.* Except for the underlying cryptographic hash functions, the implementation of our scheme relies only on mod-integer operations, which can be efficiently implemented on both software and hardware platforms, and support a high level of parallelism, that allows a flexible balance between efficiency and resources.
- *Compatibility.* We also present a key encapsulation scheme inside the COMPASS suite, both schemes are based on module-lattices with same polynomial degrees (although the moduli are different). Combining COMPASS-KEM and COMPASS-SIG allows us to achieve other functionalities, including authenticated key exchange.

Moreover, our scheme has shorter signature size than ML-DSA, so fitting our scheme into existing protocols could be easier than ML-DSA.

- *Extensibility.* Lattice-based signatures, especially those based on Fiat-Shamir with abort are known to be highly functional, by changing the underlying zero-knowledge proof, a Fiat-Shamir with abort signature scheme can handle complex functionalities, including group signature, ring signature and others. Our scheme can serve as a construction basis for those more advanced schemes.
- *Advantages of Implementation Performance across Platforms.* The basic operation unit of our scheme is polynomial in $R_q/(X^n + 1)$ for $n = 256$ or $n = 512$, which is relatively small compared with schemes based on RLWE or NTRU. While both NTT and mod-mult operations can be easily implemented in parallel, NTT operations on 256 or 512 dimensional vectors are also acceptable for the resource-restricted cases.

References

- [ACW19] Martin R Albrecht, Benjamin R Curtis, and Thomas Wunderer. Exploring trade-offs in batch bounded distance decoding. In *International Conference on Selected Areas in Cryptography*, pages 467–491. Springer, 2019.
- [ADPS16] Erdem Alkim, Léo Ducas, Thomas Pöppelmann, and Peter Schwabe. Post-quantum key exchange—a new hope. In *25th USENIX security symposium (USENIX Security 16)*, pages 327–343, 2016.
- [AFG13] Martin R. Albrecht, Robert Fitzpatrick, and Florian Goeppfert. On the efficacy of solving LWE by reduction to unique-SVP. Cryptology ePrint Archive, Paper 2013/602, 2013.
- [AGVW17] Martin R. Albrecht, Florian Goeppfert, Fernando Virdia, and Thomas Wunderer. Revisiting the expected cost of solving usvp and applications to lwe. In Tsuyoshi Takagi and Thomas Peyrin, editors, *Advances in Cryptology – ASIACRYPT 2017*, pages 297–322, Cham, 2017. Springer International Publishing.
- [BBD⁺23] Manuel Barbosa, Gilles Barthe, Christian Doczkal, Jelle Don, Serge Fehr, Benjamin Grégoire, Yu-Hsuan Huang, Andreas Hülsing, Yi Lee, and Xiaodi Wu. Fixing and mechanizing the security proof of fiat-shamir with aborts and dilithium. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology - CRYPTO 2023 - 43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA, August 20-24, 2023, Proceedings, Part V*, volume 14085 of *Lecture Notes in Computer Science*, pages 358–389. Springer, 2023.
- [CMHST25] Kevin Carrier, Charles Meyer-Hilfiger, Yixin Shen, and Jean-Pierre Tillich. Assessing the impact of a variant of matzov’s dual attack on kyber. In Yael Tauman Kalai and Seny F. Kamara, editors, *Advances in Cryptology – CRYPTO 2025*, pages 444–476, Cham, 2025. Springer Nature Switzerland.

- [CN11] Yuanmi Chen and Phong Q Nguyen. Bkz 2.0: Better lattice security estimates. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 1–20. Springer, 2011.
- [DLvW20] Léo Ducas, Thijs Laarhoven, and Wessel PJ van Woerden. The randomized slicer for cvpp: sharper, faster, smaller, batchier. In *IACR International Conference on Public-Key Cryptography*, pages 3–36. Springer, 2020.
- [EJK20] Thomas Espitau, Antoine Joux, and Natalia Kharchenko. On a dual/hybrid approach to small secret lwe: A dual/enumeration technique for learning with errors and application to security estimates of the schemes. In *International Conference on Cryptology in India*, pages 440–462. Springer, 2020.
- [EK20] Thomas Espitau and Paul Kirchner. The nearest-colattice algorithm: Time-approximation tradeoff for approx-cvp. *Open Book Series*, 4(1):251–266, 2020.
- [Kan83] Ravi Kannan. Improved algorithms for integer programming and related lattice problems. In *Proceedings of the Fifteenth Annual ACM Symposium on Theory of Computing*, STOC ’83, page 193–206, New York, NY, USA, 1983. Association for Computing Machinery.
- [KKNM25] Alexander Karenin, Elena Kirshanova, Julian Nowakowski, and Alexander May. Fast slicer for batch-cvp: Making lattice hybrid attacks practical. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 100–132. Springer, 2025.
- [KLS18] Eike Kiltz, Vadim Lyubashevsky, and Christian Schaffner. A concrete treatment of fiat-shamir signatures in the quantum random-oracle model. In Jesper Buus Nielsen and Vincent Rijmen, editors, *Advances in Cryptology - EUROCRYPT 2018 - 37th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Tel Aviv, Israel, April 29 - May 3, 2018 Proceedings, Part III*, Lecture Notes in Computer Science, pages 552–586. Springer, 2018.
- [SBN⁺20] Deepraj Soni, Kanad Basu, Mohammed Nabeel, Najwa Aaraj, Marcos Manzano, and Ramesh Karri. Crystals-dilithium. In *Hardware Architectures for Post-Quantum Digital Signature Schemes*, pages 13–30. Springer, 2020.
- [SE94] Claus-Peter Schnorr and Martin Euchner. Lattice basis reduction: Improved practical algorithms and solving subset sum problems. *Mathematical programming*, 66(1):181–199, 1994.
- [WXG25] Geng Wang, Wenwen Xia, and Dawu Gu. Heuristic algorithm for solving restricted svp and its applications. In *International Conference on Post-Quantum Cryptography*, pages 119–152. Springer, 2025.
- [XWG25] Wenwen Xia, Geng Wang, and Dawu Gu. Solve approximate cvp via variants of nearest-colattice. *Cryptology ePrint Archive*, 2025.
- [XWW⁺24] Wenwen Xia, Leizhang Wang, Geng Wang, Dawu Gu, and Baocang Wang. A refined hardness estimation of lwe in two-step mode. In *IACR International Conference on Public-Key Cryptography*, pages 3–35. Springer, 2024.